Báo cáo: Học tăng cường cho Bomberman — từ actor-critic đến chưng cất search (S3)

Oct 26, 2026 · @Danh

Tóm tắt

Agent chính của repo là **S3**, một CNN thuần (mỗi bước chỉ chạy mạng một lần rồi lấy mẫu hành động), đạt **ActiveWR 92.25%** khi đấu RuleBot trên Bomberman 9×9 1v1 (test seed 57000, 400 trận, chỉ mở một lần). Điểm xuất phát RL thuần (B) chỉ đạt 32.75%; CNN RL tốt nhất (G) đạt 49%; trần tham chiếu G + search ba nước đạt 94.0%. S3 kém trần 1.75 điểm, CI95 [−5.0, +1.5] chứa 0, nhưng nhanh hơn khoảng 70 lần.

- **Bài toán:** học một policy Bomberman biết *chủ động hạ đối thủ bằng bom của mình*, không chỉ sống sót hay thắng nhờ đối thủ tự nổ.
- **Cách làm:** actor-critic CNN có mask an toàn và teacher phụ (Phase C), rồi kill-search viết tay mô phỏng 12 tick làm "chuyên gia", rồi **expert iteration**: chưng cất dày nhãn search vào CNN, lặp G → S2 → S3.
- **Đóng góp phương pháp:** chỉ số ActiveWR tách thắng chủ động khỏi thắng thụ động; giao thức đánh giá có seed test dùng một lần, paired bootstrap theo bàn và gate chốt trước; chuỗi ablation âm tính có kiểm soát (GAE, PPO, chuẩn hóa advantage, reward shaping, curriculum, sudden death, group RL, KL neo).
- **Phát hiện chính:** (1) RL với tín hiệu giết thưa bão hòa quanh 45–50% ActiveWR; (2) search nông lúc chơi tăng mạnh (+16.5 → +21 → +8.25 điểm theo độ sâu 1/2/3 nước); (3) chưng cất dày có ví dụ ngược chuyển được gần hết lợi thế search vào CNN; (4) RL tiếp sau chưng cất xóa kỹ năng đó; (5) khi hai CNN mạnh đấu nhau, 53–64% trận hòa do cấu trúc game (mask an toàn chung), không do bỏ lỡ nước giết.
- **Giới hạn:** mỗi kết quả chính dựa trên một train seed; thầy search giả định đối thủ là RuleBot; FFA 3–4 agent và bàn 11×11 mới được đo mô tả.

1. Giới thiệu và phát biểu bài toán

Bài toán của repo là huấn luyện một agent Bomberman có kỹ năng tấn công thật: đặt bom sao cho đối thủ hết đường sống và bản thân vẫn thoát được. Đây là một Markov game cạnh tranh, phần thưởng thưa và trễ (bom nổ sau 8 tick), hành động sai một bước có thể gây tự sát.

Dự án bắt đầu theo đặc tả môn học (PROJECT_SPEC.md): Q-learning tuyến tính làm baseline, rồi actor-critic CNN theo curriculum 1v1 → self-play → FFA 4 agent. Kiểm tra thực tế ngày 10/10/2026 cho thấy các gate ban đầu đo sai thứ cần đo: checkpoint FFA 4 agent "PASS" có WR 52.5% nhưng **0 lần hạ đối thủ bằng bom của mình** trong 40 trận. Thắng chủ yếu vì RuleBot tự nổ. Từ đó bài toán được phát biểu lại như sau.

Phát biểu. Cho môi trường `BombermanFFAEnv` 9×9, 2 agent, tối đa 500 bước, sudden death lịch E (vòng co từ bước 475, cách 25 bước, cảnh báo 40 bước) và đối thủ chuẩn RuleBot cố định. Tìm policy π lấy mẫu từ một CNN, chạy một lượt mạng mỗi bước, sao cho tối đa hóa:

```
\mathrm{ActiveWR}(\pi) = \Pr_{\text{bàn},\,\text{góc}}\big[\text{learner
thắng} \wedge \text{ít nhất một đối thủ chết vì bom của learner}\big]
```

với ràng buộc tỷ lệ tự nổ không tăng quá 1 điểm phần trăm so với mốc.

Câu hỏi nghiên cứu.

1. RQ1: Các cải tiến RL chuẩn (GAE, PPO, chuẩn hóa advantage, reward shaping, curriculum) có nâng được ActiveWR của CNN khi tín hiệu giết thưa không?
2. RQ2: Search nông viết tay (mô hình đối thủ hoàn hảo) tăng ActiveWR bao nhiêu theo độ sâu, và cái giá là gì?
3. RQ3: Có chuyển được lợi thế của search vào một CNN thuần không (expert iteration / chưng cất), và cần điều kiện gì về dữ liệu?
4. RQ4: Agent mạnh trước RuleBot có còn mạnh khi đấu CNN khác, trong FFA, hoặc trên bàn 11×11 không; hòa nhiều là do agent yếu hay do luật chơi?
5. RQ5 (phương pháp): làm sao đo cải tiến một cách trung thực khi policy lấy mẫu, mỗi thí nghiệm chỉ có một train seed và phương sai giữa các seed test khoảng ±5pp?

2. Tổng quan tài liệu

Tài liệu liên quan chia thành năm nhóm. Điểm chung của nhóm Pommerman là: RL model-free thuần gần như không học được nếu không có cơ chế tránh tự sát, và các agent thắng giải đều dùng search. Repo này đi đúng con đường đó rồi nối hai nhánh lại bằng expert iteration. Phạm vi tìm kiếm: arXiv, AAAI/AIIDE, PMLR và ghi chú nghiên cứu sẵn có của repo (`RESEARCH_BOMBERMAN_2026.md`); đây là tổng quan có chọn lọc, không phải tổng quan hệ thống. Ngày 25/10/2026 đã đọc toàn văn 34/35 tài liệu trong danh mục tham khảo (Sutton & Barto chỉ đọc chương 13; Goulart 2019 chưa đọc được vì trang HAL chặn tải tự động). Insight rút ra từ toàn văn năm ở mục 2.7, ghi chú chi tiết từng bài ở tab Ghi chú đọc paper. Trang nguồn ghi ở mục Tài liệu tham khảo.

2.1 Pommerman và Bomberman

Paper	Ý chính	Liên hệ với repo
Resnick et al. 2018, Pommerman	Benchmark đa agent dựa trên Bomberman, FFA và team, đòi hỏi lập kế hoạch và mô hình đối thủ	Repo tự dựng môi trường NumPy theo thiết kế này (bàn checkerboard, gỗ, power-up, nổ dây chuyền)
Resnick et al. 2018, Backplay	Curriculum từ một demo: bắt đầu gần cuối quỹ đạo rồi lùi dần về đầu, cho môi trường thưởng thưa	Ý tưởng "bắt đầu từ trạng thái quan trọng" giống SD7 (chơi lại K lần từ trạng thái quyết định)
Peng et al. 2018, Continual Match Based Training	Huấn luyện liên tục qua các trận đấu với quần thể agent	Tương ứng Phase D (pool snapshot); repo thấy self-play làm mất kỹ năng chưng cất
Gao et al. 2019, Skynet	Hạng 2 nhóm learning agent NeurIPS 2018: reward shaping, curriculum và module suy luận cắt hành động	Cùng bộ ba công cụ Phase C của repo: shaping, curriculum, mask
Gao et al. 2019, On Hard Exploration	Khám phá ngẫu nhiên "gần như vô ích"; module model-based cắt hành động chắc chắn dẫn tới chết giúp học rõ rệt	Khớp với Phase C lần 1–5 của repo (tự nổ 84%, bomb shyness) và cách sửa bằng `_safe_action_mask`
Osogami & Takahashi 2019	Tree search thời gian thực, sau một độ sâu dùng kịch bản bi quan tất định; thắng hạng 1 và 3 NeurIPS 2018	Kill-search của repo cũng là search nông + rollout tất định (learner chỉ né), nhưng dùng mô hình RuleBot thay vì bi quan
Perez-Liebana et al. 2019	So sánh MCTS và RHEA trong Pommerman; MCTS thắng nhiều cấu hình; đề xuất mô hình đối thủ, nhận diện nước bẫy	Động lực cho SDM/SDM2 (mô hình đối thủ trong search)

Paper	Ý chính	Liên hệ với repo
Weil et al. 2023, Know your Enemy	MCTS đa người chơi quy về 1–2 người bằng mô hình đối thủ (heuristic hoặc self-play)	Giống cách `kill_search.OPPONENT_MODEL` thay đối thủ bằng RuleBot hoặc CNN
Huynh et al. 2024	PPO + curriculum đối thủ (đứng yên → di chuyển → đặt bom) + self-play quần thể và Elo, 2v2	T3 của repo thử curriculum tương tự: dừng ở C1, −20pp ActiveWR trên validation
Goulart et al. 2019	Deep RL và imitation learning cho Bomberman với biểu diễn vector (Unity ML-Agents)	Repo dùng quan sát dạng lưới 13 kênh + CNN
Sun et al., DOMAC	Actor-critic kèm dự đoán hành động đối thủ và critic phân phối, có Pommerman	Mô hình đối thủ ở mức mạng; repo đặt mô hình đối thủ ở mức search
Helfenstein et al., M2CTS	Chọn expert mạng theo giai đoạn trận rồi chạy MCTS; có thí nghiệm Pommerman FFA	Search lúc suy luận; repo cuối cùng bỏ search lúc chơi nhờ chưng cất

2.2 Search làm thầy cho mạng (search + learning)

Paper	Ý chính	Liên hệ với repo
Kartal et al. 2018, MCTS as Demonstrator và Safer Deep RL with Shallow MCTS 2019	MCTS vài rollout làm "người minh họa" không chuyên cho A3C; giảm tự sát, học nhanh hơn trên Pommerman mini 2 người	Gần nhất với SD5.1 (nhãn search trong RL). Repo cho thấy nhãn thưa (0.9/ep) không đổi được CNN (−2.75)
Kartal et al. 2019, Action Guidance	Thêm loss phụ bắt chước hành động MCTS vào actor-critic	Tương ứng các loss teacher escape/combat/navigation của repo

Paper	Ý chính	Liên hệ với repo
Anthony et al. 2017, Expert Iteration	Chuyên gia chậm (tree search) + học trò nhanh (mạng) lặp vòng; vượt REINFORCE trên Hex	**Khung chính của repo**: G → S2 → S3 là ba vòng ExIt với search viết tay
Silver et al. 2017, AlphaZero	Self-play + MCTS dẫn đường bằng mạng; mạng học phân phối lượt thăm	Repo dùng mục tiêu mềm trên các nước tốt nhất, nhưng search không dùng mạng để định hướng
Ross et al. 2011, DAgger	Gán nhãn chuyên gia trên phân phối trạng thái của chính học trò để tránh lệch phân phối	Thu dữ liệu bằng thầy = học trò + search, nên trạng thái gần với phân phối học trò
Rusu et al. 2015, Policy Distillation và Schmitt et al. 2018, Kickstarting	Chưng cất policy thầy vào học trò bằng KL/cross-entropy; thầy khởi động học trò	Loss CE mục tiêu mềm + KL neo của SD5.3/SD8A/SD9

2.3 Tối ưu policy

Paper	Ý chính	Liên hệ với repo
Sutton & Barto 2018; Williams 1992 (REINFORCE)	Policy gradient, baseline, actor-critic	Thuật toán gốc của `train_actor_critic.py` (MC returns + critic)
Mnih et al. 2016, A3C	Actor-critic bất đồng bộ, entropy bonus	Hệ số entropy và cấu trúc loss của repo
Schulman et al. 2015, GAE	Ước lượng advantage cân bằng bias–variance bằng λ	T4-B1: GAE 0.95 không hơn MC (delta 0)
Schulman et al. 2017, PPO	Mục tiêu ratio cắt ε, nhiều epoch mỗi batch	T4-D1: PPO clip 0.2 kém B (32.0 so với 37.5)
Shao et al. 2024, GRPO	Advantage = phần thưởng trừ trung bình nhóm mẫu cùng đầu vào, không cần critic	SD7 áp dụng cho game: K lần chơi lại từ cùng trạng thái quyết định

Paper	Ý chính	Liên hệ với repo
Ng et al. 1999, Reward shaping	Chỉ shaping dạng thế $\gamma\Phi(s') - \Phi(s)$ mới giữ nguyên policy tối ưu	Shaping của repo không phải dạng thế; Phase C lần 2 bị "kamikaze" đúng kiểu reward hacking

2.4 An toàn, mask và đối thủ

Paper	Ý chính	Liên hệ với repo
Alshiekh et al. 2018, Shielding	"Shield" cung cấp danh sách hành động an toàn từ đặc tả an toàn; agent có shield học ít nhất ngang, thường nhanh hơn	`_safe_action_mask` là shield kiểu "trước learner" dựa trên dự báo nổ
Huang & Ontañón 2020, Invalid Action Masking	Mask logit cho policy gradient hợp lệ; tốt hơn phạt hành động sai	Repo mask logit bằng −1e9 trước softmax
Bansal et al. 2017; Lanctot et al. 2017, PSRO	Self-play cạnh tranh, quần thể đối thủ tránh overfit	Phase D/E và D-S2 của repo

2.5 Đánh giá thống kê trong RL

Paper	Ý chính	Liên hệ với repo
Henderson et al. 2018, Deep RL that Matters	Kết quả deep RL dao động mạnh theo seed và cài đặt	Repo ghi rõ "một train seed" và đo dao động giữa seed test (KC1: ±5pp)
Agarwal et al. 2021, Statistical Precipice	Báo ước lượng khoảng (bootstrap CI) thay vì điểm	Paired bootstrap theo bàn, 10 000 lần, cho mọi so sánh

2.6 Tổng hợp

- **Điểm đồng thuận.** Trong Pommerman, khám phá ngẫu nhiên dẫn tới tự sát và "bomb shyness"; mọi agent tốt đều có một lớp cắt hành động hoặc một search. Các agent thắng giải NeurIPS 2018 là agent search, không phải RL thuần.

- **Hai dòng chưa nối.** Dòng "search lúc chơi" (Osogami, Perez-Liebana, Weil, M2CTS) mạnh nhưng chậm; dòng "search làm tín hiệu phụ trong RL" (Kartal) chỉ đo trên bản mini và đo tốc độ học. Expert iteration (Anthony, AlphaZero) nối hai dòng ở cờ bàn, nhưng chưa thấy áp dụng có đo đạc cho Bomberman.
- **Thước đo.** Các bài Pommerman báo WR hoặc điểm giải. Không bài nào được xem tách thắng do đối thủ tự nổ khỏi thắng do hạ gục, trong khi repo đo được khoảng cách này lên tới 52.5% WR so với 0% ActiveWR.
- **Mô hình đối thủ.** Weil 2023 và Perez-Liebana 2019 đều nêu mô hình đối thủ là nút thắt của search; repo đo được hệ quả cụ thể (SDM2: đoán sai loại đối thủ mất 33pp).

2.7 Insight từ đọc toàn văn

Toàn văn xác nhận ba điều mà phần tóm tắt không nói: (1) bốn nhóm tác giả độc lập (Skynet, Kartal, Huynh, Weil) đều gọi tên vấn đề "thưởng giả do đối thủ tự sát" nhưng không ai định nghĩa một thước đo tách nó ra, đây là chỗ ActiveWR của repo lấp; (2) hòa 72–89% khi hai agent mạnh gặp nhau xuất hiện ở Osogami, Perez-Liebana, Skynet và AlphaZero, nên "hòa cấu trúc" của repo là hiện tượng chung chứ không phải lỗi riêng; (3) các bài ExIt, AlphaZero, DAgger, Kickstarting giải thích được vì sao S3 dừng ở ~94% và chỉ ra bước tiếp theo cụ thể.

Pommerman, search và chưng cất

Paper	Phát hiện trong toàn văn	Hệ quả cho repo
Resnick 2018 (Pommerman)	Không có batch lớn + shaping thì DQN/PPO không thắng nổi SimpleAgent; DAgger bắt chước ~20%; agent tự rơi vào "không bao giờ đặt bom"; thi đấu dùng tường sập để phá hòa	Bomb shyness ở Phase C lần 1–5 là hiện tượng đã biết; sudden death của repo là bản 1v1 của tường sập
Backplay 2018	Bắt đầu episode gần cuối một demo rồi lùi dần; Standard PPO không học được với thưởng thưa; tổng quát 416/1000 trên bàn mới	Cùng ý với SD7 (chơi lại từ trạng thái quyết định); SD7 tăng tỉ lệ giết cục bộ nhưng chỉ ~1.2 trạng thái quyết định/trận nên không đổi ActiveWR
Peng 2018 (COMBAT)	220 CPU + 32 GPU, nhiều tuần; kỹ năng cơ bản không học sớm thì về sau không học lại được	RL thuần vượt ngân sách một laptop; khớp việc RL sau chưng cất xóa kỹ năng (D-S2)

Paper	Phát hiện trong toàn văn	Hệ quả cho repo
Gao 2019 (Skynet)	"Thưởng đánh lừa do đối thủ tự sát"; đối thủ không bom → không bao giờ vượt 20% thắng, còn lại hòa; CautiousAgent biến SimpleAgent thành rất mạnh	Động cơ trực tiếp cho ActiveWR; băng chứng độc lập cho SD9.2 (hai bên có bộ lọc an toàn thì gần như không có cú giết)
Gao 2019 (Hard Exploration)	Sau một quả bom, đi ngẫu nhiên tự sát ~40%; không cắt hành động thì PPO 0% sau 5 ngày, có cắt thì học được	Giải thích định lượng tự nổ 84% ở Phase C ban đầu; mask của repo là bản mở rộng của MinEvadeStep
Osogami 2019	Cây độ sâu 1 + kịch bản tất định bi quan ⩾10 bước; search thắng RL 87.1% / thua 2.7%; trận gương hòa 776–886/1000	Kill-search có cùng cấu trúc nhưng mô hình đối thủ là RuleBot chính xác; kịch bản bi quan là hướng chưa thử cho đối thủ CNN
Perez-Liebana 2019	MCTS với mô hình đối thủ ngẫu nhiên: thắng 46.5 / hòa 42.0; tự sát thấp nhất; đặt nhiều bom (RHEA) thì thua nhiều hơn	"Đặt nhiều bom ≠ giỏi hơn" khớp SD7.6 (bom/trận +20%, ActiveWR −0.5)
Weil 2023	"Chỉ nhìn win rate là không đủ"; expert iteration làm agent thụ động; không mô hình đối thủ đơn nào tốt cho mọi đối thủ; mô hình đối thủ bằng mạng chậm 4–9×	Khớp SDM2 (−33pp khi đoán sai loại đối thủ); nhãn search của repo chỉ +1 cho giết bằng bom của mình nên ExIt của repo không trôi về thụ động
Huynh 2024	Phạt hòa −1, đo hiệu suất bằng số địch bị hạ; 200 triệu bước vẫn thua agent search 70–95%	Sáu năm sau cuộc thi RL thuần vẫn thua search, củng cố lựa chọn search + chưng cất
Meisheri 2019	Bắt chước rồi PPO bị quên thảm họa; sửa bằng entropy 0, clip 0.01, value net tách riêng; action filter đổi thua thành hòa	Cách giữ kỹ năng chưa thử sau D-S2 và S2-KL
DOMAC 2022	Mô hình đối thủ nằm trong policy, học chỉ từ return; mô hình đúng gần cận trên dùng policy thật	Học một bộ dự đoán hành động đối thủ có thể thay việc gọi CNN thật, giảm chi phí ~53 giờ của SD10

Paper	Phát hiện trong toàn văn	Hệ quả cho repo
M2CTS 2024	Chuyên gia theo pha; cổng "mixedness" (khoảng cách tới đối thủ) giúp 5–11% trong Pommerman; chuyên gia cần đủ dữ liệu	Cổng REACH ⩽ 6 của kill-search là cùng ý; pha quyết định của repo là giao tranh giữa trận, không phải tàn cuộc
Kartal 2018/2019 (3 bài)	75 rollout giúp nhanh hơn 150 (số lượng > chất lượng); MCTS thuần an toàn nhưng hòa nhiều; hướng tương lai: gọi demonstrator ad-hoc và lưu nhãn vào buffer	Repo kiểm chứng cả hai hướng: ad-hoc trong RL thất bại (SD5.1, ~0.9 nhãn/ep), buffer + học nhiều epoch thành công (SD5.3, SD8A, SD9)
Anthony 2017 (ExIt)	Nhãn mềm TPT hơn nhãn cứng CAT +50 Elo; online (gộp dữ liệu các vòng) > batch; một trạng thái mỗi ván để giảm tương quan; chuyên gia phải dùng học trò để mạnh lên	G → S2 → S3 là ExIt batch; khoảng cách thầy–trò thu hẹp −45.75 → −3.5 → −0.75 (thầy của S3 là S2 + search3); search của repo không dùng mạng nên không có bước cải thiện chuyên gia, đó là lý do trần ~94%
AlphaZero 2017	Search dùng prior + value của mạng; không augment đối xứng với game bất đối xứng; vs Stockfish hòa 72/100	Repo bỏ augment vì 21% trạng thái đổi tập nước tốt nhất khi xoay, cùng lý do; MCTS có prior S3 là bước tiếp
Ross 2011 (DAgger)	Lỗi bắt chước tăng $T^2\varepsilon$ nếu chỉ học trên phân phối chuyên gia, DAgger đưa về tuyến tính; Mario: học trò không bao giờ thấy trạng thái lỗi của mình	Thầy = học trò + search giữ dữ liệu gần phân phối học trò; repo chưa gộp dữ liệu qua các vòng ($D_SD8A \cup D_SD9$)
Rusu 2016 (Policy Distillation)	KL với mục tiêu nhọn ($\tau = 0.01$) tốt nhất, NLL tạm, MSE kém; học trò 25% tham số đạt 108% thầy	Mục tiêu mềm chia đều các nước điểm cao nhất của repo ≈ $\text{softmax}(q/\tau)$ với $\tau \to 0$ có giữ đồng hạng; S3 đạt ~98% thầy, nhanh ~70 lần
Schmitt 2018 (Kickstarting)	λ hằng số dừng gần mức thầy (41.2), λ giảm tuyến tính 1→0 vượt thầy (59.4–60.8); nhiều thầy 73.8 vs 51.9	SD7.7 và S2-KL đều dùng β cố định và neo về chính model cũ; kickstarting gợi ý neo về thầy search với λ giảm dần

Phương pháp RL, an toàn, đa tác tử và đánh giá

Paper	Phát hiện trong toàn văn	Hệ quả cho repo
Sutton & Barto 2018, ch. 13	Dùng $\hat{v}(s)$ làm baseline không biến thuật toán thành actor-critic; actor-critic là khi $\hat{v}$ được dùng để bootstrap	Chế độ mặc định (MC return) của repo là REINFORCE với baseline học được; chỉ biến thể GAE mới bootstrap (đã sửa ở 5.2)
Williams 1992	$(r - b)\nabla \log \pi$ không chệch với mọi baseline độc lập hành động; episodic REINFORCE rất chậm vì chia đều công cho mọi bước; hàm thưởng "hiển nhiên" hay có cực đại giả	Một cú giết 1–2 bước trong ~230 bước bị pha loãng; lý do repo chuyển sang tín hiệu cục bộ (teacher, search)
Mnih 2016 (A3C)	Loss policy + value + entropy $\beta = 0.01$; cần hàng chục triệu khung hình	Cấu trúc loss của repo theo khuôn này; ngân sách 200–3000 ep/lượt thấp hơn nhiều bậc, nên "RL bão hòa" chỉ đúng ở ngân sách đó
Schulman 2016 (GAE)	$\lambda < 1$ chỉ chệch khi V sai, $\gamma < 1$ luôn chệch; tốt nhất $\lambda \in [0.92, 0.99]$; fit V trước khi tính advantage làm gradient về 0	T4-B1 (MC → GAE) không đổi ActiveWR: phương sai không phải nút thắt
Schulman 2017 (PPO)	Clip 0.2 tốt nhất (0.82); phạt KL cố định kém và khó chọn một β dùng chung	Giải thích SD7.7 (không β cố định nào tách được); nên thử β thích nghi theo KL mục tiêu
Shao 2024 (GRPO)	Advantage nhóm (r − mean)/std; online > offline; **RL tăng Maj@K nhưng không tăng Pass@K**: làm phân phối vững hơn, không thêm năng lực nền	Giải thích SD7–SD7.5: group RL tăng giết +7.1pp ở trạng thái F đã giết "lúc được lúc không" nhưng ActiveWR test chỉ +3.75; năng lực mới đến từ thầy mạnh hơn
Ng 1999	Chỉ shaping dạng thế $F = \gamma\Phi(s') - \Phi(s)$ giữ chính sách tối ưu; thưởng theo sự kiện sinh vòng lặp (xe đạp chạy vòng, robot "rung" cạnh bóng)	Thưởng phá tường/nhặt item của repo là theo sự kiện; trừ thưởng thăng thụ động là sửa mục tiêu cho khớp ActiveWR

Paper	Phát hiện trong toàn văn	Hệ quả cho repo
Alshiekh 2018 (Shielding)	Shield từ trò chơi an toàn: đúng và can thiệp tối thiểu; không phạt hành động bị sửa thì học tốt hơn	Mask của repo là shield heuristic có lớp dự phòng, chưa xét mọi nước đối thủ; margin k = 15 ở SD2 bảo thủ quá nên chỉ tăng thắng thụ động
Huang 2022 (Action Masking)	Che logit = policy gradient hợp lệ; phạt hành động sai không mở rộng được; "naive masking" kém dần theo kích thước	Đã kiểm code: repo che logit đúng kiểu đầy đủ (`masked_fill(~mask, -1e9)` cả lúc lấy mẫu và lúc tính loss)
Bansal 2018	Thưởng dày phải giảm về 0 (agent giữ thưởng dày thua agent đã giảm); đấu bản mới nhất gây mất cân bằng, nên lấy mẫu đối thủ cũ	Hệ số teacher cố định của repo có thể giới hạn trần giống thưởng dày không giảm
Lanctot 2017 (PSRO)	Policy học độc lập mất 34–72% phần thưởng khi gặp policy huấn luyện riêng (JPC); meta-strategy hỗn hợp giảm 29–57%	SDM2 là một dạng JPC; mô hình đối thủ nên là hỗn hợp {RuleBot, F, G, S2, S3}; bảng đấu chéo của repo chính là bảng lợi ích của EGTA
Henderson 2018	5 seed vs 5 seed cùng siêu tham số khác nhau có ý nghĩa ($p = 0.0016$); Swimmer "cuộn tròn quẫy" cho return đẹp mà sai hành vi	Repo làm đúng phần lớn khuyến nghị; thiếu nhiều train seed; "Swimmer" tương đương thắng thụ động
Agarwal 2021	Chế độ ít run: dùng CI bootstrap phân tầng, IQM, $P(X > Y)$; tránh kiểm định nhị phân $p < 0.05$; đổi giao thức đánh giá làm so sánh vô nghĩa	Bootstrap theo cặp bàn + gate (cỡ hiệu ứng ≥ 5pp và CI lower > 0) đúng tinh thần; CI của repo đo biến thiên do bàn, chưa đo do lần train
Efron 1979	Lấy mẫu có hoàn lại từ phân phối thực nghiệm; phải lấy lại đúng đơn vị độc lập (§7 hồi quy); thống kê hiệu cho so sánh hiệu quả hơn	Đơn vị lấy lại của repo là seed bàn (các trận cùng bàn tương quan), thống kê là hiệu ActiveWR theo cặp

Goulart 2019 (Bomberman, IL rồi PPO trong Unity) chưa đọc được toàn văn: trang HAL chặn tải tự động bằng kiểm tra chống bot, nên dòng về bài này ở mục 2.1 vẫn dựa trên tóm tắt.

3. Research gap và đóng góp

Repo lấp năm khoảng trống; khoảng trống lớn nhất là chưa có bằng chứng đo đạc rằng expert iteration với search nông viết tay đưa được một CNN thuần tới gần trần search trong Bomberman.

Khoảng trống	Tài liệu hiện có	Repo làm gì	Kết quả
G1. Thước đo không phân biệt thắng chủ động và thụ động	Skynet, Kartal, Huynh, Weil đều gọi tên "thưởng giả do đối thủ tự sát" nhưng vẫn báo WR hoặc điểm giải; Weil kết luận "chỉ WR là không đủ" mà không đưa ra thước đo thay thế	Định nghĩa ActiveWR, ghi công kill theo chủ quả bom, tách thắng thụ động / nhờ vòng / hòa / tự nổ	Phát hiện agent "PASS" có WR 52.5% mà ActiveWR 0%; loại được nhiều cải tiến giả (SD2 +10.75 WR, ActiveWR +0.75)
G2. Search + RL mới đo trên bản mini, theo tốc độ học	Kartal 2018/2019: MCTS làm demonstrator trong A3C, chỉ đường cong reward vài seed trên bàn 8×8; tự đề xuất gọi demonstrator ad-hoc và lưu nhãn vào buffer nhưng chưa thử	So sánh có đối chứng ba cách dùng search: nhãn thưa trong RL, search lúc chơi, chưng cất dày	Nhãn thưa −2.75; search lúc chơi +16.5; chưng cất dày +16.25 → +35.0 → +10.5 qua ba vòng
G3. Chưa có expert iteration cho Bomberman với search không dùng mạng	ExIt/AlphaZero cho cờ bàn, search dùng mạng dẫn đường; Weil 2023 thử ExIt trong Pommerman và agent trôi về thụ động	Search viết tay ba độ sâu làm thầy; chưng cất trạng thái có thông tin + neo KL; lặp hai lần	S3 = 92.25 ActiveWR, cách trần 94.0 có 1.75 điểm (CI chứa 0), nhanh hơn ~70 lần

Khoảng trống	Tài liệu hiện có	Repo làm gì	Kết quả
G4. Thiếu báo cáo âm tính có kiểm soát cho các kỹ thuật RL chuẩn trong môi trường kiểu này	Bài báo thường báo cấu hình thành công	Mười mấy thí nghiệm một biến, gate chốt trước, seed test mới (GAE, PPO, norm, shaping, curriculum, SD, group RL, KL neo)	Đa số FAIL; ghi đủ trong mục 7 và 9
G5. Chưa rõ hòa giữa hai agent mạnh là do agent hay do luật	Perez-Liebana, Weil: mô hình đối thủ là nút thắt; hòa cao khi agent mạnh gặp nhau (Osogami trận gương 78–89%, Skynet vs đối thủ không bom, AlphaZero vs Stockfish 72%) nhưng không bài nào tách do agent hay do luật	Probe có luật chốt trước: search đoán đối thủ bằng chính CNN của đối thủ tìm nước giết trong trận hòa	Chỉ 15.9% trận hòa có kế hoạch giết (0.19% trạng thái) → hòa do cấu trúc

Problem → solution mà repo đáp ứng.

- Tự sát và bomb shyness → mask an toàn dựa trên dự báo nổ (shield) + teacher thoát hiểm. Tự nổ 84% → 2% trên cùng trọng số.
- WR đẹp giả → ActiveWR + phân rã kết cục + reward trừ lại đúng 10 cho thăng thụ động.
- Tín hiệu giết thưa (~1.25 bước có nước giết mỗi trận 230 bước) → kill-search mô phỏng 12 tick chấm +1/−1/0 cho từng hành động.
- Search chậm (20–30 giây/trận) → chưng cất dày vào CNN, lặp expert iteration.
- Kết luận không tin cậy → giao thức đăng ký trước, seed test một lần, paired bootstrap, SHA source/checkpoint.

Độ mới cần nói rõ. Các thành phần (actor-critic, mask, search, chưng cất, group advantage) đều đã có trong tài liệu. Đóng góp nằm ở cách kết hợp cho Bomberman, thước đo ActiveWR, và bộ băng chứng đo đạc có kiểm soát (kể cả kết quả âm).

4. Môi trường

Môi trường là một Markov game cạnh tranh, tự cài bằng NumPy (`bomberman_env.py`), không phụ thuộc gym/pommerman. Với mỗi thí nghiệm, đối thủ cố định nên bài toán của learner quy về một MDP.

$$\mathcal{G} = \big(\mathcal{N}, \mathcal{S}, \{\mathcal{A}_i\}_{i\in\mathcal{N}}, P, \{r_i\}_{i\in\mathcal{N}}, \gamma\big),\qquad |\mathcal{N}| \in \{2,3,4\},\ \gamma = 0.99$$

$$\pi_{-i}\ \text{cố định} \;\Rightarrow\; P_i(s' \mid s, a_i) = \sum_{a_{-i}} \pi_{-i}(a_{-i}\mid s)\, P(s' \mid s, a_i, a_{-i})$$

Luật chơi.

- Bàn 9×9 (hoặc 11×11): viền và ô (chẵn, chẵn) là tường cứng; tường gỗ mật độ 0.7, trừ vùng 3×3 quanh góc spawn.
- 6 hành động: STOP, UP, DOWN, LEFT, RIGHT, BOMB.
- Bom nổ sau 8 tick hình dấu cộng, lửa kéo dài 2 tick, nổ dây chuyền; phá gỗ rơi power-up với xác suất 0.3 (ammo tối đa 3, range tối đa 4).
- Kill được ghi công cho chủ quả bom còn hiệu lực trúng nạn nhân; tự nổ ghi riêng.
- Sudden death lịch E: vòng ngoài đóng ở bước 475, vòng tiếp theo mỗi 25 bước, cảnh báo 40 bước trước. Tối đa 500 bước.

Quan sát 13 kênh (agent-centric, tensor 13 × H × W):

Kênh	Nội dung
0–1	Tường cứng, tường gỗ
2	Bom, giá trị = thời gian còn lại chuẩn hóa
3	Lửa
4–5	Power-up ammo, range
6–7	Vị trí bản thân, vị trí đối thủ (gộp)
8–10	Slot bom còn trống, sức chứa, tầm nổ (mặt phẳng hằng)
11	Nghịch đảo thời gian tới khi bị sát thương, tính từ `_forecast_blasts` (bom, dây chuyền, lửa còn sót)

Kênh	Nội dung
12	Cảnh báo vòng sudden death: (W − còn lại + 1) / W, bằng 0 khi tắt

Kênh 11 là nền chung cho mask an toàn, teacher và search. Checkpoint 12 kênh cũ được nạp bằng cách thêm kênh với trọng số 0, nên logits không đổi (`smart_load_weights`).

Phần thưởng. Môi trường trả cho agent i:

```
r^{\text{env}}_t = -0.001\,\mathbb{1}[\text{sống}] + 0.05\,n^{\text{item}}_t
+ 1\cdot n^{\text{kill}}_t + 10\,\mathbb{1}[\text{người sống sót duy nhất}] -
1\,\mathbb{1}[\text{chết tại } t]
```

Learner được train với phần thưởng định hình (`_shaped_reward`):

```
r^{\text{shaped}}_t = r^{\text{env}}_t + 3\,n^{\text{kill}}_t -
1.5\,\mathbb{1}[\text{tự nô}] - 10\,\mathbb{1}[\text{thăng} \wedge
K_{\text{ep}} = 0 \wedge \neg\text{SD}] + 0.15\,n^{\text{wall}}_t
```

Tổng lại: kill của mình +4, thăng chủ động +10, thăng thụ động bị trừ lại đúng 10 (còn 0), tự nổ −2.5, phá tường +0.15. Thiết kế này khớp trực tiếp với ActiveWR. Nó không phải shaping dạng thế (Ng et al. 1999) nên có thể đổi policy tối ưu; Phase C lần 2 (thưởng tường 0.3, phạt chết −0.2) đã sinh hành vi "kamikaze" phá tường rồi tự sát, WR 0.5%. Audit R0 kiểm 118 226 transition khớp công thức tới sai số 1.78e−15.

Đối thủ chuẩn RuleBot (`rule_based_bot.py`): né theo bản đồ nguy hiểm theo thời gian, đi tới gỗ/item, đặt bom khi có đường thoát. RuleBot được giữ cố định trong mọi thí nghiệm để các con số so được với nhau. Hai RuleBot đấu nhau 100 ván: WR 36% / 48%, hòa 16%, không crash (Phase A, gate PASS).

5. Methodology: công thức, phương pháp, thuật toán

Phương pháp cuối cùng gồm ba tầng: actor-critic CNN có shield và teacher phụ tạo ra CNN nền; kill-search viết tay làm chuyên gia; chưng cất dày lặp lại (expert iteration) đưa chuyên gia vào CNN. Mỗi mục dưới ghi: công thức, lý do dùng, giải quyết được gì, tốt đến đâu (số liệu từ mục 7).

5.1 Baseline: Q-learning với xấp xỉ tuyến tính (Phase B)

$$\hat{Q}(s,a;\mathbf{w}) = \mathbf{w}_a^{\top}\boldsymbol{\phi}(s), \qquad \mathbf{w}_a \leftarrow \mathbf{w}_a + \alpha\big[r + \gamma \max_{a'}\hat{Q}(s',a') - \hat{Q}(s,a)\big]\boldsymbol{\phi}(s)$$

- **Đặc trưng:** 10 rồi 16 đặc trưng thủ công (khoảng cách tới gỗ, bom, đối thủ, số ô an toàn, lối thoát…), ε-greedy giảm dần, α = 0.005, bàn 7×7.
- **Lý do:** baseline theo môn học (feature construction, TD).
- **Tốt đến đâu:** WR 8% → 39.8% (lần 1), đỉnh 56% nhưng không ổn định (lần 2); bom giảm từ 2.2 xuống 0.2–0.4/ván (bomb shyness). Giới hạn năng lực của xấp xỉ tuyến tính là lý do chuyển sang CNN.

5.2 Policy gradient CNN (Phase C): REINFORCE có baseline, tùy chọn GAE/PPO

Mạng. Conv 3×3 (13→32) → Conv (32→64) → Conv (64→64), ReLU, flatten 64·H·W → FC 256 → hai đầu: actor (6 logit) và critic V(s). Tham số θ dùng chung.

Policy có mask. Với mask an toàn m(s) ∈ {0,1}^6:

$$\pi_\theta(a\mid s) = \frac{m_a(s)\,\exp z_a(s)}{\sum_{b} m_b(s)\,\exp z_b(s)}\qquad (\text{cài đặt: } z_a \leftarrow -10^{9} \text{ khi } m_a = 0)$$

Mục tiêu (REINFORCE với baseline học được). Mỗi episode, return được tính ngược, advantage = return trừ giá trị critic. Code gọi là "actor-critic" theo thói quen A2C, nhưng theo định nghĩa của Sutton & Barto (ch. 13) đây là REINFORCE với baseline V(s) học bằng Monte Carlo: critic chỉ làm baseline giảm phương sai, không bootstrap. Theo Williams (1992) gradient này không chệch với mọi baseline không phụ thuộc hành động. Chỉ biến thể GAE (5.5) mới bootstrap như actor-critic đúng nghĩa:

$$G_t = \sum_{k=t}^{T} \gamma^{\,k-t} r^{\text{shaped}}_k,\qquad \hat{A}_t = G_t - V_\theta(s_t)$$

$$\mathcal{L}(\theta) = \underbrace{-\tfrac{1}{T}\sum_t \log\pi_\theta(a_t\mid s_t)\,\hat{A}_t}_{\text{policy}} + c_v\,\underbrace{\tfrac{1}{T}\sum_t \big(V_\theta(s_t)-G_t\big)^2}_{\text{critic}} - c_e(e)\,\bar{\mathcal{H}}[\pi_\theta] + \mathcal{L}_{\text{aux}}$$

$$c_v = 0.5,\qquad c_e(e) = c_e^{0}\cdot\max\!\big(0.2,\; 1 - 0.8\,e/E\big),\qquad c_e^{0} = 0.0075$$

Phần phụ trợ (teacher, search, group, neo) ở các mục dưới:

$$\mathcal{L}_{\text{aux}} = \lambda_{esc}\mathcal{L}_{esc} + \lambda_{cmb}\mathcal{L}_{cmb} + \lambda_{nav}\mathcal{L}_{nav} + \lambda_{ring}\mathcal{L}_{ring} + \lambda_{srch}\mathcal{L}_{srch} + \lambda_{grp}\mathcal{L}_{grp} + \beta\,\mathrm{KL}(\pi_\theta \,\|\, \pi_{\text{ref}})$$

Mặc định (công thức của bạn cùng nhóm): lr 2.5e−5 (Adam), λ_esc 0.5, λ_cmb 0.9, λ_nav 0.4, λ_ring 0.6, λ_srch 0.9; các pilot cũ dùng lr 1e−4. Luân phiên góc học mỗi episode. Chọn checkpoint bằng validation định kỳ (ActiveWR cao nhất với tự nổ không tăng), không dùng WR cuộn lúc train.

- **Lý do:** policy gradient trực tiếp trên không gian hành động rời rạc nhỏ; critic giảm phương sai; CNN khai thác cấu trúc lưới. Entropy giảm dần giữ khám phá đầu trận và ổn định cuối.
- **Tốt đến đâu:** đưa ActiveWR từ ~0% lên 32.75% (B), rồi ~48–51% (F, G). Sau đó bão hòa: train thêm 3000 ep từ F không tăng (control − F gộp ba seed test 0.0 [−3.75, +3.75]).

5.3 Mask an toàn dựa trên dự báo nổ (shield)

`_forecast_blasts` mô phỏng trước mọi quả bom (kể cả dây chuyền, fuse bị rút ngắn, lửa còn sót) và cho ra thời điểm bị sát thương d(c) của từng ô c. Một hành động a được phép nếu sau a vẫn tồn tại đường đi tới ô an toàn trước hạn:

$$m_a(s) = \mathbb{1}\Big[\exists\ c_0{=}\text{pos}(s,a), c_1, \dots, c_k:\ \ d(c_j) > j\ \ \forall\ j \le k,\ \ d(c_k) = \infty\Big]$$

Các điều kiện bổ sung: với BOMB thì dự báo gồm cả quả bom mới; tránh hành lang một lối ra mà đối thủ tới được trong 2 bước; tránh ô kề đối thủ; có biên 2 tick trước phản bom. Ngoại lệ "bom bổ sung": cho đặt bom khi đang trong tia dự báo nếu learner vẫn sống được và đối thủ mất toàn bộ vị trí sống ở tick nổ. RuleBot dùng cùng mask (bản không có ngoại lệ).

- **Lý do:** giống shielding (Alshiekh 2018) và action pruning (Gao 2019): khám phá ngẫu nhiên trong Bomberman chủ yếu sinh tự sát, gradient học "đừng đặt bom". Mask logit cho policy gradient hợp lệ (Huang & Ontañón 2020).
- **Giải quyết:** tự sát và bomb shyness.

- **Tốt đến đâu:** cùng trọng số, tự nổ 84% → 20% và ActiveWR 0% → 23% (100 trận seed 7000); bản cuối tự nổ 30% → 7% (seed 12000). Ngoại lệ bom bổ sung: ActiveWR 9.5% → 31% trên 200 trận seed 14000, tự nổ giữ 2%, +43 thắng chủ động, không mất trận nào.
- **Cái giá:** mask chung cho cả hai bên là nguyên nhân cấu trúc của hòa khi hai CNN đấu nhau (SD9.2): không bên nào bước vào vùng nổ.

5.4 Teacher phụ (auxiliary supervision)

Mỗi bước, teacher heuristic gán nhãn một hành động a* (hoặc phân phối đích q) cho một trong bốn nhóm, lấy trung bình riêng từng nhóm để nhãn chiến đấu hiếm không bị nhãn điều hướng lấn át:

```
\mathcal{L}_g = -\frac{1}{|\mathcal{T}_g|}\sum_{t\in\mathcal{T}_g} \sum_a
q_t(a)\,\log\pi_\theta(a\mid s_t)\cdot \big(1.75\ \text{nếu } t \ge 350\
\text{và } g = cmb\big),\quad g\in\{esc, cmb, nav, ring\}
```

- **esc:** đang ở ô nguy hiểm → bước đầu của BFS tới ô an toàn gần nhất.
- **cmb (v2, pressure/sequence):** chọn ô đặt bom làm giảm nhiều nhất số vị trí sống của đối thủ ở tick nổ (ngưỡng lợi áp lực 0.15, chuỗi 2 bước 0.35).
- **nav:** đi tới gỗ/item/vị trí tấn công; **ring:** rời ô săp bị vòng đóng.
- **Lý do:** giống Action Guidance (Kartal 2019): bơm tri thức miền vào khi phần thưởng thưa.
- **Tốt đến đâu:** cần thiết để có B và F, nhưng các biến thể teacher mạnh hơn đều không qua gate: two_bomb 0 [0, 0], MOVE guidance +5.25 [−1.5, +12]. SD6 cho thấy teacher escape gán đúng một lối thoát `exits[0]` có thể là nguyên nhân F bỏ chạy thay vì đứng giữ ô cửa (giả thuyết, chưa chứng minh). D-S2 cũng nghi nhãn teacher dày kéo model chưng cất về hành vi cũ.

5.5 Biến thể ước lượng advantage và mục tiêu (T4)

GAE (Schulman 2015), không bootstrap qua terminal:

```
\delta_t = r_t + \gamma\,V(s_{t+1})\,\mathbb{1}[\neg\text{terminal}] -
V(s_t),\qquad \hat{A}^{\text{GAE}}_t = \sum_{l\ge 0}
(\gamma\lambda)^l\,\delta_{t+l},\quad \lambda = 0.95
```

Chuẩn hóa advantage theo quỹ đạo: Â ← (Â − mean) / std (bỏ qua khi std < 1e−8).

PPO (Schulman 2017), 4 epoch mỗi episode, dừng sớm theo KL (target 0.02, ngưỡng 0.03):

$$\rho_t(\theta) = \frac{\pi_\theta(a_t\mid s_t)}{\pi_{\theta_{\text{old}}}(a_t\mid s_t)},\qquad \mathcal{L}^{\text{CLIP}} = -\mathbb{E}_t\Big[\min\big(\rho_t \hat{A}_t,\ \mathrm{clip}(\rho_t, 1-\epsilon, 1+\epsilon)\,\hat{A}_t\big)\Big],\ \ \epsilon = 0.2$$

- **Lý do:** giảm phương sai (GAE, chuẩn hóa) và tăng hiệu quả mẫu (PPO) là các cải tiến RL chuẩn nhất.
- **Tốt đến đâu:** không cái nào qua gate. GAE − B 0 [0, 0] (best vẫn là episode 0); chuẩn hóa − B +0.25 [−5.5, +6]; PPO 32.0% so với B 37.5%. Kết luận: nút thắt không nằm ở bộ ước lượng gradient mà ở chỗ tín hiệu giết quá thưa.

5.6 Sudden death và biên an toàn (SD1–SD4)

Vòng co đóng dần ô viền từ bước start, mỗi interval bước một vòng. Kênh 12 cảnh báo trước W = 40 bước. Mask vòng của learner cấm ô sẽ bị đóng trong k bước tới:

$$m^{\text{ring}}_a(s) = m_a(s)\cdot\mathbb{1}\big[\tau_{\text{close}}(\text{pos}(s,a)) > t + k\big],\qquad k = 15$$

- **Lý do:** ép trận kết thúc, giảm hòa 500 bước.
- **Tốt đến đâu:** tăng WR nhưng không tăng ActiveWR. Biên k = 15: WR +10.75 [6.25, 15.5], ActiveWR +0.75 [−1.25, 2.75], chết vì vòng −10pp. Ablation SD8B-off: với G + search3, tắt sudden death lúc chơi cho 399/400 trận giống hệt.

5.7 Kill-search: chuyên gia viết tay (`kill_search.py`)

Với mỗi hành động hợp lệ a, sao chép môi trường và mô phỏng H = 12 tick (8 tick bom + 2 tick lửa + dư): learner làm a ở tick đầu rồi chỉ né (BFS tới ô an toàn, nếu không thì STOP); đối thủ chơi theo mô hình đối thủ M (mặc định RuleBot, tất định). Điểm:

$$\mathrm{score}(s,a) = \begin{cases} +1 & \text{bom của learner giết đối thủ khi learner còn sống, trong } H \text{ tick}\\ -1 & \text{learner chết}\\ 0 & \text{còn lại} \end{cases}$$

Chỉ gọi search khi có đối thủ trong khoảng Manhattan ≤ 6. Giữ hành động ã do CNN lấy mẫu, trừ khi có hành động điểm cao hơn hẳn:

$$a^{\text{exec}} = \begin{cases} \tilde{a} \sim \pi_\theta(\cdot\mid s) & \text{nếu } \mathrm{score}(s,\tilde{a}) \ge \max_b \mathrm{score}(s,b)\\ \text{BOMB nếu đạt max, ngược lại } \min\{\arg\max_b \mathrm{score}(s,b)\} & \text{ngược lại} \end{cases}$$

Search nhiều nước. Khi không nước đơn nào giết, search2 thử mọi nước đầu an toàn rồi mọi nước thứ hai hợp lệ; search3 đệ quy thêm một mức. Kế hoạch ngắn được thử trước:

$$V_d(s) = \begin{cases} \max_b \mathrm{score}(s,b) & d = 1 \ \text{hoặc}\ \max_b \mathrm{score}(s,b) > 0\\ \max\Big(\max_b \mathrm{score}(s,b),\ \max_{b:\,\mathrm{score}(s,b)\ge 0} \mathbb{1}\big[V_{d-1}(T(s,b)) > 0\big]\Big) & d > 1 \end{cases}$$

với T(s,b) là trạng thái sau một tick (learner làm b, đối thủ theo M) và horizon giảm 1 mỗi mức. Hành động trả về là nước đầu của một kế hoạch giết. RNG của game thật được lưu và khôi phục quanh search.

- **Lý do:** tín hiệu giết cực thưa (~2.2 bước có thông tin, ~1.25 bước có nước giết mỗi trận 230 bước). Mô phỏng với mô hình đối thủ đúng cho biết chính xác nước nào giết. Cách chấm chỉ +1 cho kill bằng bom của learner khớp ActiveWR, nên không tạo thăng thụ động.
- **Giải quyết:** bỏ lỡ cửa sổ giết 1 bước (31/37 cửa sổ thất bại dài 1 bước), điểm mù "đứng giữ ô cửa", và với search2/3 là *tạo* cơ hội giết.
- **Tốt đến đâu:** G + search: +16.5 [+12.5, +20.5] (test 46000); search2 so với search: +21.0 [+17.0, +25.25] (test 47000); search3 so với search2: +8.25 [+5.25, +11.5] (test 49000), đạt 94.0%. Search một nước có trần ~65% bất kể nền CNN.
- **Cái giá:** ~1.1 s/trận (CNN) so với ~24 s (search3) đấu RuleBot, ~130 s khi đấu CNN; chỉ mạnh khi M đúng loại đối thủ.

5.8 Advantage theo nhóm từ trạng thái quyết định (SD7, kiểu GRPO)

Khi kill-search thấy có nước giết ở trạng thái s (bộ lọc: có bom đang cháy, đối thủ trong vùng nguy hiểm và cách ⩽ 4), chơi lại từ s K lần bằng policy hiện tại, mỗi lần H = 20 tick với RNG riêng. Kết cục $o_k \in \{+1, -1, 0\}$ theo cùng quy ước với kill-search:

$$A_k = o_k - \frac{1}{K}\sum_{j=1}^{K} o_j,\qquad \mathcal{L}_{grp} = -\frac{1}{N}\sum_{\text{nhóm có tương phản}}\ \sum_{k=1}^{K}\ \sum_{(s,a)\in\tau_k} A_k\,\log\pi_\theta(a\mid s)$$

Nhóm mà mọi o_k bằng nhau bị bỏ (không có tín hiệu). SD7.6 chỉ lấy trạng thái "hẹp" có 1–2 nước giết và chỉ học bước đầu, có buffer chơi lại. SD7.7 thêm neo KL về G:

```
\mathrm{KL}(\pi_\theta\,\|\,\pi_G)(s) = \sum_a \pi_\theta(a\mid
s)\big[\log\pi_\theta(a\mid s) - \log\pi_G(a\mid s)\big]
```

- **Lý do:** giống GRPO (Shao 2024) và Backplay: baseline là trung bình nhóm nên không cần critic; chỉ hành động của learner phân biệt các lần chơi lại, nên gán công chính xác. Spike SD7.0: ở trạng thái có nước giết, F giết được 60%, 86% trạng thái "lúc được lúc không".
- **Tốt đến đâu:** cơ chế hoạt động khi tín hiệu đủ mạnh (coef 4, K 16: tỷ lệ giết +7.12 [+4.86, +9.72]) nhưng không thành ActiveWR cả trận (SD7.5: +3.75 [−2.5, +10.0], FAIL). Lý do định lượng: ~1.2 trạng thái quyết định/trận × 7pp ≈ 1pp, nhỏ hơn CI ±6pp. SD7.6 làm policy "cứ được đặt bom là đặt" (P(BOMB) 0.64 → 0.95, bom/trận +20%); neo KL với mọi $\beta \in \{0, 1, 4, 16\}$ không tách được hai việc. Vẫn sinh ra **G** (SD7.5 best ep 1000), CNN RL tốt nhất theo lựa chọn của người dùng.

5.9 Chưng cất dày và expert iteration (SD5.3, SD8A, SD9)

Thu dữ liệu. Thầy = học trò hiện tại + kill-search (độ sâu 1/2/3) chơi 3000 trận đấu RuleBot (1500 bàn × 2 góc, lịch E với xác suất 0.75). Ghi lại:

- **Trạng thái quyết định** D: search chạy và điểm các hành động không bằng nhau (gồm cả ví dụ ngược "ở đây bom không phải nước tốt nhất").
- **Trạng thái neo** N: 5% trạng thái còn lại, kèm logits của model gốc.

Mục tiêu mềm. Khối xác suất chia đều cho các hành động hợp lệ đạt điểm cao nhất:

```
B(s) = \arg\max_{a:\,m_a(s)=1} \mathrm{score}(s,a),\qquad y_a(s) =
\frac{\mathbb{1}[a\in B(s)]}{|B(s)|}
```

Loss học trò (bắt đầu từ model gốc π_{ref}, batch 256 nửa D nửa N):

```
\mathcal{L}_{\text{distill}}(\theta) = \underbrace{-
\mathbb{E}_{s\sim\mathcal{D}}\sum_a y_a(s)\log\pi_\theta(a\mid s)}_{\text{CE
mục tiêu mềm}} +
\lambda\,\underbrace{\mathbb{E}_{s\sim\mathcal{N}}\,\mathrm{KL}\big(\pi_{\tex
t{ref}}(\cdot\mid s)\,\|\,\pi_\theta(\cdot\mid s)\big)}_{\text{neo hành vi
thường}},\quad \lambda = 1
```

Adam lr 1e−4; 8 epoch (SD5.3) hoặc 16 epoch với ứng viên 4/8/12/16 (SD8A, SD9); held-out 10% bàn. Chọn epoch trên validation 21000 (ActiveWR cao nhất, tự nổ ≤ mốc + 1pp) trước khi mở test. Không tăng cường dữ liệu bằng đối xứng vì search và RuleBot không đối xứng qua phép quay (79% trạng thái giữ nguyên tập nước tốt nhất).

Vòng expert iteration:

1. π_0 = G (CNN từ RL).
2. Thầy $T_k = \pi_k$ + search_d; thu D_k, N_k bằng T_k trên dải bàn mới.
3. π_{k+1} = argmin L_distill bắt đầu từ π_k; chọn epoch bằng validation; test một lần trên seed mới.
4. Lặp: G + search2 → S2 (SD8A, bàn 210000–211499); S2 + search3 → S3 (SD9, bàn 212000–213499, 10 425 trạng thái quyết định, 15 865 neo).

- **Lý do:** expert iteration (Anthony 2017) tách "tìm nước tốt" (search chậm) khỏi "khái quát" (mạng nhanh). Thu bằng chính học trò + search giữ phân phối trạng thái gần học trò (tinh thần DAgger). Chỉ học trạng thái có thông tin, nhiều epoch, có ví dụ ngược; KL neo trên trạng thái thường giữ hành vi cũ (policy distillation / kickstarting).
- **Giải quyết:** RL không đổi được CNN vì nhãn thưa và một phía (SD5.1, SD7.x); search chậm lúc chơi.
- **Tốt đến đâu:** S − F +16.25 [+9.75, +22.5] (test 43000); S2 − G +35.0 [+29.0, +41.25] (test 48000); S3 − S2 +10.5 [+6.5, +14.5] (test 57000). Held-out: P(nước tốt nhất) của F 0.52 → S 0.91, nhóm "đứng giữ ô cửa" 0.28 → 0.93, trong khi P(BOMB) ở trạng thái thường gần như không đổi (0.088 → 0.095). Chi phí train chỉ 17 giây GPU; phần đắt là thu dữ liệu (4 giờ 24 phút cho SD9).

5.10 Mô hình đối thủ trong search (SDM, SDM2)

`kill_search.OPPONENT_MODEL` thay RuleBot bằng hàm M(env, i) bất kỳ. Hai biến thể được đo: *oracle* = argmax policy đã mask của chính CNN đối thủ; *self* = luôn đoán bằng CNN G.

```
a_i^{\text{model}} = \arg\max_a\ m_a(s)\,\pi_{\phi_i}(a\mid o_i(s))
```

- **Tốt đến đâu:** oracle khi đấu G +14.0 [+3, +25], đấu S +4.0 [−6, +15], chậm ~2.4 lần. Self: đấu CNN F +12.0 [+3, +22] nhưng đấu RuleBot **−33.0 [−41, −25]**. Mô hình đối thủ phải đúng loại đối thủ; không mô hình đơn nào tốt cho mọi đối thủ. Mặc định giữ RuleBot.
- **SD10 (đã cài, chưa chạy):** chưng cất S3 + search3 đoán đối thủ đúng khi thu dữ liệu đấu CNN. Đo chi phí: ~628 s mỗi trận đấu CNN cho ~2.7 nhãn, tổng ~53 giờ.

6. Giao thức thí nghiệm và đánh giá

Mọi so sánh chính dùng cùng một giao thức: chọn trên validation, test một lần trên seed mới 400 trận mỗi model, so ghép cặp theo bàn bằng bootstrap, gate chốt trước.

6.1 Chỉ số

Với tập trận 🧱 (bàn b, góc c ∈ {A0, A1}):

$$\mathrm{ActiveWR} = \frac{1}{|\mathcal{S}|}\sum_{(b,c)\in\mathcal{S}} \mathbb{1}\big[\text{thắng}_{b,c} \wedge \text{kills}^{\text{own}}_{b,c} > 0\big],\qquad \mathrm{WR} = \mathrm{ActiveWR} + \mathrm{PassiveWR} + \mathrm{RingWR}$$

Kèm theo: hòa (tách hòa hết 500 bước), thua, tự nổ, chết vì vòng, kills/trận, bom/trận, số bước, số liệu theo góc. WR không được dùng làm chỉ số chính vì nó tăng khi đối thủ tự nổ hoặc chết vì vòng.

6.2 So sánh ghép cặp và khoảng tin cậy

Hai model chơi cùng 200 bàn, mỗi bàn cả hai góc. Chênh lệch theo bàn và percentile bootstrap (resample bàn, giữ hai góc cùng nhau; B = 10 000; seed 20261014):

$$d_b = \tfrac{1}{2}\sum_{c\in\{0,1\}}\big(y^{\text{cand}}_{b,c} - y^{\text{ref}}_{b,c}\big),\qquad \hat{\Delta} = \frac{100}{n}\sum_{b=1}^{n} d_b$$

$$\hat{\Delta}^{*(j)} = \frac{100}{n}\sum_{b=1}^{n} d_{I_{jb}},\ \ I_{jb}\sim\mathcal{U}\{1..n\},\qquad \mathrm{CI}_{95} = \big[Q_{0.025}(\hat{\Delta}^{*}),\ Q_{0.975}(\hat{\Delta}^{*})\big]$$

Ghép cặp theo bàn khử phương sai do bản đồ (một số bàn khó với mọi model). Macro CNN (SD9.1, S3-11) lấy trung bình chênh lệch qua ba đối thủ trước khi bootstrap.

6.3 Gate chốt trước

Một ứng viên thay model hiện tại khi cùng lúc:

$$\hat{\Delta}_{\text{ActiveWR}} \ge 5\ \text{pp},\qquad Q_{0.025}(\hat{\Delta}^{*}_{\text{ActiveWR}}) > 0,\qquad \Delta_{\text{tự nổ}} \le 1\ \text{pp}$$

Một số thí nghiệm thêm điều kiện "không kém control". Probe chỉ trên validation có luật quyết định riêng (ví dụ SD9.2: ≥ 50% trận hòa có nước giết → hướng A; ≤ 20% → cân nhắc đổi luật).

6.4 Quản lý seed và nguồn gốc

- **Validation:** seed 21000 (100–400 trận); chẩn đoán 6600, 6800, 6850, 6900, 15000.
- **Test dùng một lần:** 16000, 17000, 25000, 26000, 32000–42000, 44000–50000, 51000–56099, 57000, 60000, 820000–820199, 830000–830199. Không chạy thêm test để "cứu" gate; không chọn checkpoint sau khi xem test.
- **Đăng ký trước:** runner ghi `registration.json` (config, SHA256 source và checkpoint) trước khi train; `selected.json` ghi trước `test_opened.json`; từ chối ghi đè thư mục output; chép source vào output.
- **Tái lập:** lưu toàn bộ trạng thái train (model, Adam, scheduler, RNG Python/NumPy/env/Torch) để resume chính xác. Policy lấy mẫu nên chỉ tái lập bit-to-bit trên cùng phiên bản torch (cùng F: 47.0 với torch 2.5.1, 43.75 với 2.14.1).

6.5 Độ nhạy của giao thức

Cùng một cặp model (control SD5.1 so với F) cho −6.5, +3.5, +3.0 trên ba seed test; gộp 600 cặp bàn thì 0.0 [−3.75, +3.75] (KC1). Vậy một seed test 400 trận dao động khoảng ±5pp, đó là lý do ngưỡng gate là +5pp. Chọn best trong nhiều điểm validation cũng thiên lạc quan vài pp (control SD5.1: validation 51.0, test 42.0).

7. Danh sách thí nghiệm

Repo có hơn 50 thí nghiệm ghi trong `RESULTS.md`; chỉ bảy thay đổi qua được gate ActiveWR trên test (search một, hai, ba nước lúc chơi; ba vòng chưng cất S, S2, S3; chưng cất S3-11 trên 11×11). Điều kiện mặc định trừ khi ghi khác: đấu RuleBot, 9×9, 1v1, max_steps 500, policy lấy mẫu + mask, 400 trận/model (200 bàn × 2 góc), paired bootstrap 10 000 lần seed 20261014, CPU, torch 2.5.1. Delta là điểm phần trăm ActiveWR, ngoặc vuông là CI95.

7.0 Các checkpoint mốc

Ký hiệu	Đường dẫn	SHA256 rút gọn	Nguồn gốc
B	`runs/pressure-C-next/final_phaseC.pt`	`93bdf278…4f60`	Phase C 500 ep từ `combat_bc_phaseC.pt`, lr 1e−4, seed 2026101301
F	`runs/phaseC-sd-best-fallback/best_phaseC.pt`	`0315a56f…1483`	~3000 ep từ B (1500 + 750 + 750), nhánh bạn cùng nhóm, 13 kênh
G	`runs/sd75-20261018/full/best_phaseC.pt`	`bb0c775c…c7ac`	SD7.5 best ep 1000 (group RL từ F)
S	`runs/sd53-20261018/train/epoch_8.pt`	`8f7116d2…fd4a`	Chưng cất F + search1 (SD5.3)
S2	`runs/sd8a-20261020/train/epoch_16.pt`	`d222d81f…6814`	Chưng cất G + search2 (SD8A)
S3	`runs/sd9-20261023/train/epoch_12.pt`	`4185375f…69ed7`	Chưng cất S2 + search3 (SD9), agent chính
S3-11	`runs/s3-map11-20261024-auto-v1/…/epoch_16`	`17748471…a95f`	Chưng cất S3 + search3 trên 11×11

7.1 Giai đoạn theo đặc tả ban đầu (A–E, 03–05/10)

Các số này đo trên cửa sổ train hoặc vài chục trận, trước khi có ActiveWR và giao thức test riêng; chỉ dùng để mô tả đường đi.

Mã	Thông số	Kết quả	Gate
A	RuleBot vs RuleBot, 11×11, 100 ván	WR 36/48%, hòa 16%, 11.7 bom/ván	PASS
B lần 1	Q-learning tuyến tính, 7×7, 5000 ep, lr 0.005, 10 đặc trưng	WR 500 ep cuối 39.8%; bom 2.2 → 0.4	FAIL (cần ≥ 50%)
B lần 2	8000 ep, 16 đặc trưng, thưởng tường +0.1	Đỉnh 56%, 500 ep cuối 41.0%; bom 0.2	PARTIAL

Mã	Thông số	Kết quả	Gate
C lần 1	AC CNN, 9×9, 3000 ep, lr 3e−4, thưởng tường 0.08, RTX 4060	WR 70%, **0 bom/ván**	FAIL bomb shy
C lần 2	6000 ep, thưởng tường 0.3, phạt chết −0.2	WR 0.5%, 4.3 bom: "kamikaze"	FAIL
C lần 3–5	6000/6000/4000 ep	WR 55–65.5%, 0 bom	FAIL bomb shy
C lần 6	+ mask an toàn, giám sát thoát, 4000 ep	WR 56% (đỉnh 88% ở ep 2550–2700), 4.3 bom	FAIL (overtraining)
C lần 7	+ best checkpoint, cosine lr 3e−4 → 1.5e−5, entropy 0.015 → 0.003	Best WR 82.5%, final 70%, 1.7 bom	PASS
D lần 1	Self-play pool, 4000 ep, rule_prob 0.2	Round-robin 67.2%, vs RuleBot 38%	FAIL (quên)
D lần 2	3000 ep, rule_prob 0.5, phạt tự sát −2.5	vs RuleBot 85.5%, clean WR vs snapshot 65.8%	PASS
E1 / E2	FFA 9×9 3 agent 2000 ep; 11×11 4 agent 3000 ep (nội suy song tuyến 9→11)	Reward tăng; thắng theo góc 47/21/10.5/21%	PASS
Kiểm tra hạ gục (10/10)	`best_phaseE_4a.pt` vs 3 RuleBot, 11×11, 40 ván	WR 52.5%, **ActiveWR 0%**, 0 kill	Phát hiện gate sai

7.2 Sửa môi trường, mask và đánh giá Phase C (10–13/10)

Mã	Thông số	Kết quả	Quyết định
Phase C 3000 ep (người dùng)	9×9, CPU, từ đầu	Tự nổ 84.2%, ActiveWR 3.7%	Phát hiện lỗi mask (fuse dây chuyền, BOMB khi STOP bị cấm)
Sửa mask, cùng trọng số	100 trận seed 7000	Tự nổ 84 → 20%, ActiveWR 0 → 23%	Giữ code mới

Mã	Thông số	Kết quả	Quyết định
RuleBot mới + trần range	100 trận seed 7000	ActiveWR 3% (range 4) / 4% (range 5)	Chọn range 4
Phase C mới 3000 ep	Kill credit mới	Best ActiveWR 6%, tự nổ 21%, hòa 67% (100 trận)	Luân phiên góc, chọn bằng validation
Pilot thoát bom 500 ep	lr 1e−4, seed 20261011, RTX 3050; test 100 trận seed 9000	ActiveWR 8% cả ba cấu hình; tự nổ 25 → 10%	Giữ checkpoint gốc
Chẩn đoán giao tranh + warm-up teacher	602 nhãn bom + 6958 nhãn điều hướng, 200 update, lr 5e−5; test 100 trận seed 12000	ActiveWR 7 → 9%, tự nổ 30 → 6%	Chọn `combat_bc_phaseC.pt`
Thưởng tường 0.15 / 0.03	500 ep mỗi lượt; 200 trận seed 13000	Thắng có hạ gục 11 → 12	Không cải thiện
Ngoại lệ bom bổ sung	Cùng trọng số; 200 trận seed 14000	ActiveWR 9.5 → **31%**, tự nổ 2 → 2%	Giữ
Train 500 ep → **B**	lr 1e−4, thưởng tường 0.15, seed 2026101301; 200 trận seed 15000	ActiveWR 23.5 → 25%	B trở thành mốc

7.3 Tinh chỉnh RL quanh B (T1–T4, 14–15/10)

Tất cả từ B, 200 episode/nhánh, Phase C vs RuleBot, lr 1e−4, γ 0.99, teacher local, CUDA RTX 3050, validation 100 trận seed 21000 ở ep 0/100/200.

Mã	Biến đổi	Seed test	ActiveWR (B / control / ứng viên)	Delta ứng viên − B	Gate
T1	Teacher hai bom vs local	26000	best two_bomb = B (ep 0)	0 [0, 0]	FAIL
T2	Audit nhãn combo (không train)	28000	final two_bomb kém B	−17 [−25.5, −8.5]	Giữ B

Mã	Biến đổi	Seed test	ActiveWR (B / control / ứng viên)	Delta ứng viên − B	Gate
T3.2	Curriculum kỹ năng (C1 static) vs control 600 ep	val 21000	curriculum 18% vs control 38%	−20 [−32, −8] (val)	Dừng ở C1
Control best400	Xác nhận control T3.2	32000	32.75 / — / 35.0	+2.25 [−3.26, +8]	FAIL
MOVE guidance	`local_combat_move`	33000	35.0 / 35.5 / 40.25	+5.25 [−1.5, +12]	FAIL (CI)
T4-A1	Chuẩn hóa advantage	34000	33.25 / 36.5 / 33.5	+0.25 [−5.5, +6]	FAIL
T4-B1	GAE λ 0.95 vs MC	35000	best = B	0 [0, 0]	FAIL
T4-R1	Thưởng tường 0.075 vs 0.15	36000	34.5 / 36.25 / 35.0	chưa đạt +5	FAIL
T4-D1	PPO clip 0.2 vs ratio U4, 4 epoch, KL target 0.02	37000	37.5 / 37.0 / 32.0	−5.5	FAIL

7.4 Sudden death và luật lúc chơi (SD1–SD6, 16–18/10)

Mã	Thông số	Kết quả chính	Quyết định
SD1	B, không train; start 200/interval 20/warning 40; 4 case × 400 trận seed 38000	off: WR 38.75, Active 32.75; both_aware: WR 43.0, Active 17.5	Vòng sớm hại ActiveWR
SD2	Quét timing trên validation 21000, 200 trận/cấu hình; chọn 400/40/40, max_ring 2	both_aware: WR 59.25, Active 31.5 (off 35.5)	WR tăng nhờ thăng thụ động
SD3	Train B dưới SD2 ± kênh vòng, 200 ep/nhánh, seed 2026101601; test 40000	sd_12ch − B −2.25 [−8.0, 3.5]; sd_13ch −2.25 [−7.5, 3.25]	FAIL, giữ B
SD4.1	Biên mask vòng k ∈ {0,5,10,15,20} trên val; test k 0 vs 15 seed 41000	WR +10.75 [6.25, 15.5]; Active +0.75 [−1.25, 2.75]; chết vì vòng −10pp	PASS gate riêng; ActiveWR không đổi
SD4.2a	Từ bước 360 giao ghế learner cho teacher local / two_bomb / RuleBot; val 21000	Đều kém B: −4.25, −3.5, −2.25	Dừng hướng SD cuối trận

Mã	Thông số	Kết quả chính	Quyết định
SD4.6	B + kill-search (H 12) bật từ bước 0 / 360; val 21000	search@0 **+5.75 [2.75, 8.75]**; search@360 −0.25	Mở hướng search
XC	F vs B dưới lịch E; seed 31000 (200) + 60000 (400)	Gộp 600 trận: F − B **+7.5 [1.83, 13.17]**	Chuyển sang F
SD5.0	F + search từ bước 0, val 21000	Active 47.0 → 69.5 (+22.5 [18.25, 26.75])	Mở SD5.1
SD5.1	Từ F, 3000 ep/nhánh, lr 2.5e−5, nhãn search hệ số 0.9, SD 75% ep; test 42000	search − F −2.75 [−9.75, 4.0]; control − F −6.5 [−12.75, 0.0]	FAIL, giữ F
SD5.3a	Mật độ thông tin, F + search 20 trận seed 6600	230 bước/trận, 2.2 bước có thông tin, 1.25 có nước giết	Thiết kế chưng cất dày
SD6.0	Nhiệt độ lấy mẫu τ ∈ {1, 0.5, 0.25, argmax}, val 21000	argmax **−9.25 [−16.0, −2.5]**; τ 0.25 +1.75 [−5.5, 8.75]	Phải lấy mẫu
SD6.1b	Luật đứng giữ ô cửa (cutoff) lúc chơi, val 21000	WR +3.75; Active −1.75 [−4.25, 0.75]; thụ động 9.5 → 15.25	Không train

7.5 Advantage theo nhóm (SD7, 18–19/10)

Từ F (hoặc G với SD7.6/7.7), train seed 2026101701, lr 2.5e−5, entropy 0.0075, combat 0.9, SD lịch E ở 75% episode, CUDA RTX 3050.

Mã	Cấu hình	Đo cơ chế (tỷ lệ giết ở trạng thái quyết định)	ActiveWR test	Quyết định
SD7.0 spike	F, K 8, H 20, seed 6600	F giết 60.2%, 86% trạng thái lúc được lúc không	—	Mở SD7
SD7.1–7.3	coef 1, K 8, 3000 ep	group − control +0.34 [−1.03, 1.71]	seed 44000: group − F +0.25 [−5.75, 6.25]	FAIL

Mã	Cấu hình	Đo cơ chế (tỷ lệ giết ở trạng thái quyết định)	ActiveWR test	Quyết định
SD7.4	coef 4, K 16, 1000 ep, không val	group − control **+7.12 [+4.86, +9.72]**	—	H1′ ĐẠT
SD7.5	coef 4, K 16, 3000 ep, 4 h 58 ph	—	seed 45000: full − F +3.75 [−2.5, 10.0]; full − control +0.75	FAIL; người dùng chọn best ep 1000 làm **G**
SD7.6a	Chẩn đoán, seed 6800, 336 trạng thái	31/37 cửa sổ thất bại dài 1 bước; 28 là "đặt bom ngay", F gán P ≈ 0.58	—	Nhắm khoảnh khắc bom
SD7.6-probe	narrow + buffer, từ G, 1000 ep, seed 6850	buffer − control **+18.43 [+14.58, +22.28]**	val: Active −0.5, bom/trận +20%	Lan "đặt bom bừa"
SD7.7-calibrate	Neo KL về G, β ∈ {0,1,4,16}, 150 ep	P(BOMB) 0.64 → 0.83–0.95 ở mọi β	—	Không tách được, dừng
KC1	Gộp control − F trên 42000/44000/45000	—	0.0 [−3.75, +3.75]	Dao động ±5pp mỗi seed

7.6 Search lúc chơi và chưng cất (19–23/10)

Mã	Thông số	Kết quả	Gate
SD7.8	G vs G + search1, test 46000, 5 tiến trình	48.25 → **64.75**, +16.5 [+12.5, +20.5]; thụ động −0.5	ĐẠT

Mã	Thông số	Kết quả	Gate
SD5.3 (bạn chạy)	F + search1 thu 3000 trận bàn 200000–201499, 8 epoch, lr 1e−4, batch 256, λ 1, RTX 4060, torch 2.14.1; test 43000	F 44.25 → S **60.5**, +16.25 [+9.75, +22.5]; F + search 64.25	ĐẠT
SD7.9a	G+search, S, S+search; val 21000	66.25 / 64.0 / 65.5	Trần search1 ~65%
SD7.9c	G + search1 vs G + search2, test 47000	61.0 → **82.0**, +21.0 [+17.0, +25.25]; hòa 18 → 5.75	ĐẠT
SD8A (bạn chạy)	G + search2 thu 3000 trận bàn 210000–211499; 16 epoch, lr 1e−4, batch 256, λ 1; RTX 4060, torch 2.11; test 48000	G 48.0 → S2 **83.0**, +35.0 [+29.0, +41.25]; S2 − S +25.5; S2 − (G + search2) −3.5 [−8.25, 1.25]	ĐẠT
SD8B	G + search2 vs G + search3; val 21000 và test 49000; ~23 s/trận	85.75 → **94.0**, +8.25 [+5.25, +11.5]; hòa 4.75 → 0.25	ĐẠT
SD9.0	S2 + search3 vs G + search3, val 21000, 100 trận/cặp	vs RuleBot −5.0 [−11, 1]	Thầy SD9 = S2 + search3
SD9	S2 + search3 thu 3000 trận bàn 212000–213499 (10 425 quyết định, 15 865 neo); 16 epoch, ứng viên 4/8/12/16, lr 1e−4, batch 256, λ 1, RTX 3050, 17 s; chọn epoch 12 trên val (90.5); test 57000	S2 81.75 → S3 **92.25**, +10.5 [+6.5, +14.5]; S3 − (G + search3) −1.75 [−5.0, +1.5]; S3 − G +43.25	ĐẠT, S3 là agent chính
D-S2 (bạn chạy)	Phase D từ S2, 2000 ep/nhánh, control RuleBot vs pool CNN	Active vs RuleBot 83.75 → 65.0 / 57.5; đúng nước search 97.4 → 75.6 / 73.1%	VALIDATION_STOP

Mã	Thông số	Kết quả	Gate
S2-KL (bạn chạy)	500 ep đầu RuleBot từ S2, β 0 vs 1	R: anchor 77.5 vs control 75.0; Active anchor 68 vs S2 84	RB_RETENTION_FAIL

7.7 Đấu CNN, FFA và bàn 11×11 (20–25/10)

Mã	Thông số	Kết quả	Kết luận
PDE	Seed 50000, lịch E; G + search3 vs RuleBot trong FFA; 1v1 vs CNN 200 trận/cặp	FFA 9×9 3 agent Active 94.44; 11×11 4 agent 86.25. 1v1 vs CNN: Active 8.5–20.5, WR 19–44, hòa 50–70%	Chỉ đánh giá với RuleBot đánh giá quá cao search
SDM	Oracle (CNN thật của đối thủ) trong search3, val 21000, 100 trận	vs G +14.0 [+3, +25]; vs S +4.0 [−6, +15]	"Giúp" theo luật
SDM2	Luôn đoán bằng CNN G	vs F +12.0; vs S +9.0; vs RuleBot **−33.0 [−41, −25]**	KHÔNG GIỮ
PDE2	Bàn 51000–56099; FFA hỗn hợp, gương, đối đầu	FFA 9×9 hòa 73.5%, G + s3 Active 9.0, S2 5.0; 11×11 hòa 88%	Đa số hòa khi agent mạnh đấu nhau
SD9.1	S3 vs S2 đấu S2/G/S3, val 21000, 400 trận/cặp	Macro CNN Active +3.33 [+1.25, +5.50]; hòa −4.08; S3 vẫn hòa 53–64%	KHÔNG ĐẠT
SD9.2	Trong 195 trận hòa của S3, search3 đoán đối thủ bằng CNN thật	0.19% trạng thái, **15.9% [9.9, 22.7]** trận hòa có kế hoạch giết (giả định RuleBot: 90.3%)	Hòa do cấu trúc
SD9.3	Lịch vòng S400/S350/S300 (interval 25) so với E	Macro CNN Active −0.17 / −1.42 / −0.50; hòa thành cùng chết ở vòng 3 (80–87%)	Giữ E

Mã	Thông số	Kết quả	Kết luận
S3-MAP11	Baseline 11×11 50 bàn: S3 50%, S3 + search3 98%. Thu 200 trận bàn 812000–812099 (746 quyết định, 1310 neo), 16 epoch từ S3, CUDA; test 820000–820199	S3 nội suy 46.25 → S3-11 **87.25**, +41.0 [34.5, 47.25]; tự nổ +0.5	ĐẠT; giữ riêng cho 11×11
S3-MAP11-CNN	11×11, S3-11 vs S3 nội suy đấu S3/S2/G, 2400 trận, bàn 830000–830199	Macro Active 8.58 vs 4.75, +3.83 [1.92, 5.83]; hòa 64.33 vs 63.08	Chưa đạt +5pp
SD10	Chưng cất S3 + search3 đoán đối thủ đúng; bàn 214000–214999, test 58000	Code và test xong; thu ước tính ~53 giờ	Chưa chạy

7.8 Chuyển S3 sang bàn 11×11 (chi tiết)

Trên 11×11, chưng cất lại đúng bàn mới nâng S3 từ 46.25% lên **87.25% ActiveWR** khi đấu RuleBot (+41.0 [34.5, 47.25]). Khi đấu CNN, mức tăng chỉ còn +3.83 [1.92, 5.83], chưa đạt ngưỡng +5pp.

Thông số chung: 11×11, 13 kênh, 1v1, max_steps 500, lịch E 475/25/40, CNN lấy mẫu + mask. S3 gốc (`4185375f…69ed7`) được nạp lên 11×11 bằng nội suy trọng số (`smart_load_weights`). Chạy trên Windows, Python 3.11.9, torch 2.11.0+cu128, NumPy 2.4.6 (Codex thực hiện, kiểm tra 45 phút một lần). Run `runs/s3-map11-20261024-auto-v1` và `runs/s3-map11-cnn-20261025-v1` .

Bước 2, baseline (50 bàn 21000–21049 × 2 góc = 100 trận/agent, đấu RuleBot, CPU 2 worker, ~40 phút):

Agent	WR	ActiveWR	Hòa	Tự nổ	Chết vì SD
S3 nội suy lên 11×11	62%	50%	32%	0%	6%
S3 + search3 (thầy)	100%	98%	0%	0%	0%

Thầy − S3: ActiveWR +48 [+38, +58]. Vì thầy còn hơn xa trên đúng bàn mới, nên đáng thu nhãn mới thay vì tin rằng trọng số 9×9 đã đủ.

Bước 3–4, thu và train: thầy S3 + search3 thu 200 trận (bàn 812000–812099 × 2 góc, lịch E xác suất 0.75, neo 5%, CPU 4 worker, ~41 phút) được 746 trạng thái quyết định và 1310 neo; thầy đạt ActiveWR 98% trên tập thu. Train từ S3: Adam 1e−4, batch 256, λ 1, 16 epoch, ứng viên 4/8/12/16, RNG 530, held-out 10% bàn (59 quyết định, 149 neo), CUDA RTX 4060. Held-out: chọn đúng nước tốt nhất 67.80% (epoch 0) → 81.36% (epoch 16).

Bước 5, validation (200 bàn 21000–21199 × 2 góc = 400 trận/model; luật: ActiveWR cao nhất với tự nổ ⩽ S3 + 1pp):

Model	WR	ActiveWR	Hòa	Tự nổ	Chết vì SD
S3 nội suy	60.00%	45.00%	30.25%	0.00%	8.75%
epoch 4	87.50%	78.25%	9.50%	0.50%	2.00%
epoch 8	89.00%	82.00%	6.50%	1.25% (loại)	2.00%
epoch 12	90.50%	86.00%	6.75%	0.50%	1.25%
epoch 16 (chọn)	91.75%	87.00%	4.25%	1.00%	2.25%

Test một lần, đấu RuleBot (200 bàn 820000–820199 × 2 góc = 400 trận/model, paired bootstrap theo bàn 10 000 lần):

Chỉ số	S3 nội suy, chưa học 11×11	S3-11 epoch 16	Delta [CI95]
WR	59.25%	90.75%	+31.50 [25.75, 37.25]
ActiveWR	46.25%	**87.25%**	**+41.00 [34.50, 47.25]**
Hòa	30.75%	6.00%	−24.75 [−30.0, −19.5]
Tự nổ	0.75%	1.25%	+0.50 [−1.0, 2.0]
Chết vì SD	6.75%	1.50%	−5.25
Thăng thụ động	2.00%	2.00%	0

Gate (ActiveWR ⩾ +5pp, CI dưới > 0, tự nổ ⩽ +1pp): **PASS**. Phần tăng là thăng chủ động; thăng thụ động giữ nguyên 2%.

Đấu pool CNN (S3-11 và S3 nội suy cùng đấu S3/S2/G đã chuyển lên 11×11; bàn 830000–830199 × 2 góc = 400 trận/cặp, 6 cặp = 2400 trận, CPU 5 worker, ~23 phút; macro = trung bình ba đối thủ trước khi bootstrap):

Chỉ số (macro)	S3-11	S3 gốc chuyển map	Delta [CI95]
WR	23.00%	21.67%	+1.33 [−1.83, 4.50]
ActiveWR	**8.58%**	4.75%	**+3.83 [1.92, 5.83]**
Hòa	64.33%	63.08%	+1.25 [−2.50, 5.08]
Hòa hết bước	61.58%	62.17%	−0.58 [−4.42, 3.33]
Thua	12.67%	15.25%	−2.58 [−5.25, 0.00]
Tự nổ	1.58%	1.33%	+0.25 [−0.75, 1.17]
Thắng thụ động	0.75%	0.67%	+0.08 [−0.67, 0.75]
Thắng nhờ vòng	13.67%	16.25%	−2.58 [−5.25, 0.08]
Chết vì vòng	13.75%	13.08%	+0.67 [−2.08, 3.33]

Theo từng đối thủ (400 trận mỗi dòng):

Learner	Đối thủ	WR	ActiveWR	Hòa	Hết bước	Thua	Tự nổ
S3-11	S3	26.25	9.50	62.25	60.50	11.50	1.50
S3-11	S2	19.75	8.75	64.25	62.00	16.00	2.00
S3-11	G	23.00	7.50	66.50	62.25	10.50	1.25
S3 nội suy	S3	21.25	5.75	57.50	57.00	21.25	2.50
S3 nội suy	S2	17.50	4.25	68.25	67.00	14.25	1.00
S3 nội suy	G	26.25	4.25	63.50	62.50	10.25	0.50

Ngưỡng báo cáo (macro ActiveWR ≥ +5pp, CI dưới > 0, tự nổ ≤ +1pp): **chưa đạt** vì ước lượng điểm +3.83 < 5, dù CI dương. Hòa vẫn 62–66%, phần lớn hết 500 bước, giống hiện tượng hòa cấu trúc trên 9×9 (SD9.2). Không train hay đổi model phục vụ.

Giới hạn: một lượt train; pilot nhỏ (200 trận thu, 746 nhãn so với 10 425 nhãn của SD9); đối thủ CNN là model 9×9 chuyển map, chưa có CNN train native 11×11; chưa kiểm tra S3-11 còn giữ kỹ năng 9×9 hay không. FFA đo sau ở mục 7.9. S3-11 được giữ riêng cho 11×11; S3 vẫn là agent chính 9×9.

7.9 FFA của S3 và S3-11 (26/10)

S3 chỉ được chưng cất từ ván 1v1 nhưng vẫn giữ kỹ năng giết khi chơi FFA 3 agent trên bàn 9×9: **ActiveWR 82.8%**, hơn G **+46.7 điểm [+36.1, +56.7]** trên cùng bộ bàn. Trên 11×11 với 4 agent, nội suy S3 sang bàn lớn hầu như mất hết kỹ năng (25.0%). Còn S3-11, được chưng cất ngay trên 11×11, đạt **59.4%**.

Thông số: `run_s3_ffa_check.py`, block mới seed 59000 (dùng một lần), lịch E, 500 bước, mỗi learner đấu RuleBot. E1 = 9×9, 3 agent, 60 bàn × 3 góc = 180 trận; E2 = 11×11, 4 agent, 40 bàn × 4 góc = 160 trận. G chạy lại trên cùng bàn để so ghép cặp. Bootstrap theo bàn 10 000 lần, seed 20261014. Output `runs/s3-ffa-59000/`. Chỉ mô tả, không có gate.

Bối cảnh	Learner	ActiveWR % [CI95]	WR %	Hòa %	Tự nổ %	Thắng chủ động / tổng thắng
E1 9×9, 3 agent (mức chia đều 33%)	RuleBot (mốc)	7.2 [3.9, 10.6]	16.7	50.0	33.9	43%
E1	G	36.1 [28.3, 44.4]	56.1	32.2	3.3	64%
E1	**S3**	**82.8 [76.1, 88.9]**	87.8	7.2	0.6	**94%**
E2 11×11, 4 agent (mức chia đều 25%)	RuleBot (mốc)	4.4 [1.9, 7.5]	13.1	47.5	48.8	33%
E2	G (nội suy)	17.5 [11.9, 23.8]	30.0	60.6	3.1	58%
E2	S3 (nội suy)	25.0 [18.1, 31.9]	33.8	52.5	3.8	74%
E2	**S3-11**	**59.4 [50.6, 68.1]**	63.8	25.0	5.0	**93%**

So ghép cặp ActiveWR (pp): E1 S3 − G +46.7 [+36.1, +56.7]; E2 S3 − G +7.5 [−1.9, +16.9] (CI chứa 0); E2 S3-11 − S3 +34.4 [+23.8, +44.4]; E2 S3-11 − G +41.9 [+31.9, +51.9].

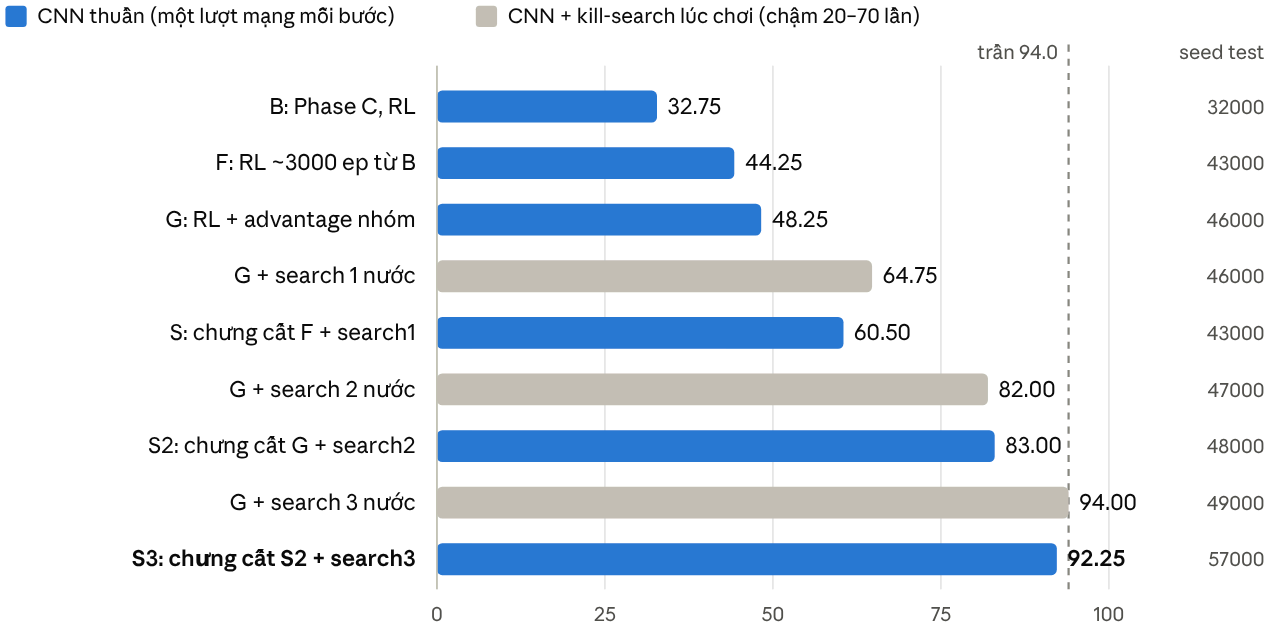
- **FFA ủng hộ ActiveWR.** Trong FFA, đối thủ còn chết vì bom của chính nó, vì nhau hoặc vì vòng, nên WR thổi phồng agent yếu nhiều nhất. G thắng 56.1% ở E1 nhưng 20 điểm trong số đó không do G giết; ở S3 và S3-11, 93–94% trận thắng là tự giết.
- **Đổi kích thước bàn thì chưng cất lại, đừng nội suy** (INSIGHTS mục 11).
- **Khoảng cách tới G+s3 lớn hơn ở 1v1:** 9×9 FFA S3 82.8 so với G+s3 94.4; 11×11 S3-11 59.4 so với G+s3 86.3 (G+s3 đo ở block 50000, khác block, nên chỉ mô tả).
- **Giới hạn:** cùng một G nhưng block 50000 cho 27.2, block 59000 cho 36.1, tức một block 160–180 trận dao động khoảng ±9 điểm; mỗi bối cảnh chạy một lượt; G+s3 không chạy lại trên 59000 (khoảng 11 giờ CPU).

8. Experiment path: từ con số 0 đến agent tốt nhất

CNN thuần đi từ ActiveWR 0% (checkpoint "PASS" theo đặc tả ban đầu) lên 32.75 (B), 48.25 (G) bằng RL, rồi lên 92.25 (S3) bằng ba vòng chưng cất search. Mỗi vòng, search lúc chơi đi trước và CNN chưng cất đuổi theo gần sát.

CNN thuần tăng từ 32.75 lên 92.25 ActiveWR nhờ chưng cất search

ActiveWR (%) đấu RuleBot, 9×9 1v1, 400 trận trên seed test riêng; xếp theo thời gian



RESULTS.md · 9 mốc, mỗi mốc 400 trận trên một seed test dùng một lần (seed khác nhau nên so trực tiếp giữa các mốc chỉ mang tính mô tả)

1. **03–05/10, theo đặc tả.** Q-learning tuyến tính (WR ≈ 40%, bomb shy) → actor-critic CNN; mask an toàn và best checkpoint giúp Phase C–E "PASS" theo WR.
2. **10/10, đổi thước đo.** Kiểm tra hạ gục: WR 52.5% nhưng 0 kill. Định nghĩa ActiveWR, sửa ghi công kill, sửa mask (tự nổ 84% → 20%), thêm reward trừ thăng thụ động.
3. **11–13/10, xây B.** Validation cố định, luân phiên góc, warm-up teacher, ngoại lệ bom bổ sung (ActiveWR 9.5 → 31%), train 500 ep → B.
4. **14–15/10, RL quanh B thất bại.** Teacher hai bom, curriculum, MOVE guidance, chuẩn hóa advantage, GAE, reward, PPO: không cái nào qua gate.
5. **16–17/10, sudden death và phát hiện search.** SD tăng WR nhưng không tăng ActiveWR. Kill-search lúc chơi là cách đầu tiên tăng ActiveWR (+5.75 trên B, +22.5 trên F). Chuyển sang F của bạn cùng nhóm (+7.5 so với B).
6. **17–19/10, thử đưa search vào RL.** Nhãn search thưa trong RL (SD5.1) FAIL; advantage theo nhóm (SD7) tăng tỷ lệ giết cục bộ nhưng không tăng ActiveWR; chọn G.

7. **19–20/10, search làm trần.** G + search 1/2/3 nước: 64.75 → 82.0 → 94.0. Song song, SD5.3 chứng minh chưng cất dày chuyển được search vào CNN (S, 60.5).
8. **20–23/10, expert iteration.** G + search2 → S2 (83.0); S2 + search3 → S3 (92.25). RL tiếp từ S2 (D-S2, S2-KL) làm mất kỹ năng nên dừng.
9. **23–25/10, kiểm tra ngoài phân phối.** Đầu CNN vẫn hòa nhiều vì cấu trúc (SD9.1–SD9.3); chưng cất riêng cho 11×11 (S3-11, +41.0). Chốt S3 là agent chính ngày 24/10.

9. Ablation study

Ba thành phần quyết định kết quả là mask an toàn, độ sâu search và việc chưng cất *dày* thay vì nhãn thưa; các thành phần RL chuẩn (GAE, PPO, chuẩn hóa, reward) và sudden death gần như không đổi ActiveWR. Mỗi dòng chỉ đổi một biến so với đối chứng ghi trong cột so sánh.

9.1 Thành phần huấn luyện và an toàn

Thành phần	So sánh	Δ ActiveWR [CI95]	Seed, số trận	Kết luận
Mask an toàn (sửa dự báo)	code cũ vs mới, cùng trọng số	0 → 23% (tự nổ 84 → 20%)	7000, 100	Thiết yếu
Ngoại lệ bom bổ sung trong mask	tắt vs bật, cùng trọng số	9.5 → 31% (+43 / −0 trận)	14000, 200	Thiết yếu
Thưởng phá tường	0.075 vs 0.15	chênh < 2pp, FAIL	36000, 400	Không nhạy
Ước lượng advantage	GAE 0.95 vs MC	0 [0, 0]	35000, 400	Không nhạy
Chuẩn hóa advantage	có vs không	−3 [−9.25, 3.25] so với control	34000, 400	Không giúp
Mục tiêu PPO	clip 0.2 vs ratio không clip	32.0 vs 37.0	37000, 400	Không giúp
Teacher hai bom	two_bomb vs local	−1.5 [−7.25, 4.5]	26000, 400	Không giúp
MOVE guidance	có vs local	+4.75 [−1.5, 11]	33000, 400	Chưa đủ bằng chứng

Thành phần	So sánh	Δ ActiveWR [CI95]	Seed, số trận	Kết luận
Curriculum kỹ năng (C1)	curriculum vs control cùng 200 ep	−20 [−32, −8]	val 21000, 100	Có hại
Train dưới sudden death	sd_12ch vs B	−2.25 [−8.0, 3.5]	40000, 400	Không giúp
Kênh quan sát vòng	13 vs 12 kênh	0.0 [−4.75, 4.75]	40000, 400	Không giúp
Lấy mẫu vs argmax	argmax vs τ = 1	**−9.25 [−16.0, −2.5]**	val 21000, 400	Phải lấy mẫu
Nhiệt độ thấp	τ 0.25 vs 1	+1.75 [−5.5, 8.75]	val 21000, 400	Không nhạy

9.2 Search lúc chơi

Thành phần	So sánh	Δ ActiveWR [CI95]	Seed	Kết luận
Có search (1 nước)	G + search vs G	**+16.5 [+12.5, +20.5]**	46000	Thiết yếu
Độ sâu 2	search2 vs search1	**+21.0 [+17.0, +25.25]**	47000	Đòn bẩy lớn nhất: tạo cơ hội giết
Độ sâu 3	search3 vs search2	**+8.25 [+5.25, +11.5]**	49000	Vẫn có lợi, gần trần
Thời điểm bật search	chỉ từ bước 360 vs từ bước 0 (nền B)	−0.25 vs +5.75	val 21000	Trận được quyết định trước bước 475
Nền CNN cho search1	S + search vs G + search	65.5 vs 66.25	val 21000	Search1 có trần ~65% bất kể nền
Nền CNN cho search3	S2 + s3 vs G + s3 (vs RuleBot)	−5.0 [−11, 1]	val 21000, 100	Nền mạnh hơn không giúp

Thành phần	So sánh	Δ ActiveWR [CI95]	Seed	Kết luận
Sudden death lúc chơi (search3)	lịch E vs off	0.0 [0, 0], 399/400 trận giống hệt	49000	Chỉ phân xử ván dài
Sudden death lúc chơi (search2)	E vs off	0.0 [−1, 1]; WR +3.25, hòa −4	49000	WR đổi, ActiveWR không
Mô hình đối thủ (đầu RuleBot)	đoán bằng CNN G vs RuleBot	**−33.0 [−41, −25]**	val 21000, 100	Phải đúng loại đối thủ
Mô hình đối thủ (đầu G)	oracle CNN vs RuleBot	+14.0 [+3, +25]	val 21000, 100	Giúp khi đúng
Luật đứng giữ ô cửa	cutoff vs không	−1.75 [−4.25, 0.75]	val 21000	Tăng WR thụ động

9.3 Cách đưa search vào CNN

Cách	So sánh	Kết quả	Seed	Kết luận
Nhãn search thưa trong RL (~0.9 nhãn/ep, học một lần)	SD5.1 search vs F	−2.75 [−9.75, 4.0]	42000	Không đổi được CNN
Advantage nhóm yếu	coef 1, K 8 vs control	tỷ lệ giết +0.34 [−1.03, 1.71]	seed 6650	Không đủ tín hiệu
Advantage nhóm mạnh	coef 4, K 16 vs control	tỷ lệ giết +7.12; ActiveWR test +0.75	6700 / 45000	Cục bộ có, toàn trận không
Nhóm "hẹp" + buffer	SD7.6 vs control	tỷ lệ giết +18.43; ActiveWR val −0.5; bom +20%	6850 / 21000	Lan "đặt bom bừa"
Neo KL trong group RL	$\beta \in \{0, 1, 4, 16\}$	P(BOMB) và P(nước giết) tăng như nhau	6600	Không tách được

Cách	So sánh	Kết quả	Seed	Kết luận
Chưng cất dày có ví dụ ngược	S vs F; S2 vs G; S3 vs S2	**+16.25; +35.0; +10.5** (CI đều > 0)	43000 / 48000 / 57000	Cách duy nhất hiệu quả
Số epoch chưng cất (SD9)	epoch 4/8/12/16 vs S2	+10.0 / +11.25 / +11.5 / +11.25	val 21000	Bão hòa từ epoch 8
RL tiếp sau chưng cất	D-S2 2000 ep vs S2	Active vs RuleBot 83.75 → 57.5–65	val	Xóa kỹ năng
Neo KL toàn cục khi RL tiếp	β 1 vs 0, 500 ep	R +2.5pp; Active 68 vs 76	val	Không giữ được
Chưng cất đúng bàn 11×11	S3-11 vs S3 nội suy	**+41.0 [34.5, 47.25]**	820000	Nội suy trọng số không đủ

9.4 Luật chơi (đấu CNN)

Biến đổi	Δ macro CNN ActiveWR	Δ hòa	Kết luận
Vòng từ bước 400 (S400)	−0.17 [−1.25, 1.00]	−3.25	Không giúp
S350	−1.42 [−2.42, −0.42]	−0.92	Hại nhẹ
S300	−0.50	−2.92	Không giúp; hòa thành cùng chết ở vòng 3

Đọc ablation. ActiveWR đạt được nhờ thông tin về nước giết (từ search), không nhờ cách tối ưu gradient. Bất kỳ thay đổi nào không mang thêm thông tin đó (GAE, PPO, shaping, SD, teacher heuristic) đều nằm trong ±5pp. Thông tin đó chỉ vào được CNN khi được cung cấp dày, có ví dụ ngược, học nhiều epoch có giám sát.

10. Phát hiện, thảo luận, giới hạn và hướng tiếp

10.1 Phát hiện (potential findings)

1. **WR là thước đo dễ bị đánh lừa trong Bomberman.** Nhiều can thiệp tăng WR 4–11 điểm mà ActiveWR đứng yên (SD2, biên k = 15, luật ô cửa). Ghi công kill theo chủ bom cho thấy cả luật "làm đối thủ chết" cũng có thể biến thăng chủ động thành thụ động (SD6.1b đổi 16 trận). Trả lời RQ5.
2. **RL model-free bão hòa vì thiếu thông tin, không vì thuật toán.** Mỗi trận ~230 bước chỉ có ~2.2 bước mà lựa chọn hành động thay đổi kết cục; 31/37 cửa sổ giết thất bại chỉ dài 1 bước. GAE/PPO/chuẩn hóa không tạo ra thông tin này. Trả lời RQ1: không.
3. **Độ sâu search quan trọng hơn nền CNN.** Search1 có trần ~65% với mọi nền; search2 phá trần nhờ *tạo* cơ hội (đi một bước rồi đặt bom): 87 trận chuyển thành thắng chủ động, 3 trận mất, trung vị bước thắng 205 → 158. Trả lời RQ2.
4. **Chưng cất dày chuyển được search vào CNN; RL một phía thì không.** Điểm khác biệt là ví dụ ngược: nhãn chưng cất nói cả "ở đây đặt bom" lẫn "ở đây bom không tốt", nên P(BOMB) ở trạng thái thường giữ ~0.09, trong khi group RL đẩy P(BOMB) lên 0.95 mọi nơi. Trả lời RQ3: có, với điều kiện nhãn dày, hai phía, nhiều epoch.
5. **Expert iteration hội tụ nhanh về trần của thầy.** Khoảng cách CNN − thầy: G −45.75 → S2 −3.5 → S3 −0.75 [−4.0, +2.5] so với thầy S2 + search3 (CI chứa 0; so với trần G + search3 là −1.75). Thêm một vòng cùng loại thầy khó qua gate +5pp vì chỉ còn 1–2 điểm.
6. **Kỹ năng chưng cất dễ vỡ dưới RL.** Chỉ 500 ep RL đã làm "đúng nước search" rơi 97.4 → 75–77.5%; drift tập trung ở trạng thái quyết định (KL lớn gấp ~10 lần trạng thái thường), nên neo KL toàn cục bị pha loãng.
7. **Hòa giữa hai agent mạnh là tính chất cấu trúc.** Khi cả hai dùng chung mask an toàn, gần như không có cú giết chắc chắn (0.19% trạng thái); bẫy kiểu RuleBot là ảo khi đấu CNN (90.3% → 15.9%). Vòng co sớm hơn chỉ biến hòa thành cùng chết. Trả lời RQ4.
8. **Mô hình đối thủ trong search phải khớp loại đối thủ.** Sai loại mất 33pp; đúng loại thêm 12–14pp. Khớp với nhận định của Weil 2023 và Perez-Liebana 2019.

10.2 Thảo luận so với tài liệu

- Kết quả ủng hộ nhận định của Gao et al. (2019) rằng khám phá model-free trong Pommerman gần như vô ích nếu không cắt hành động chết: mask là bước tăng lớn đầu tiên của repo.

- Khác Kartal et al. (2019), dùng search làm tín hiệu phụ trong RL không đủ ở đây (SD5.1). Lý do khả dĩ: họ đo tốc độ học từ đầu trên bản mini, còn repo cần vượt một policy đã bão hòa; nhãn của repo thưa hơn nhiều (0.9/ep).
- Phù hợp với Anthony et al. (2017): tách lập kế hoạch và khái quát thành hai tác tử hiệu quả hơn RL thuần. Khác AlphaZero, search của repo không dùng mạng dẫn đường nên vòng lặp không tự nâng trần thầy; đây là lý do S3 hội tụ về ~94%.
- GRPO-style (Shao 2024) hoạt động về cơ chế, nhưng trong game có rất ít trạng thái quyết định mỗi trận thì hiệu ứng toàn trận nhỏ hơn nhiều đánh giá. Toàn văn của Shao cho thêm một lý do: RL tăng Maj@K nhưng không tăng Pass@K, tức chỉ làm vững những gì policy đã làm được; đúng với SD7.4 (tăng giết ở trạng thái F đã giết được 86% "lúc được lúc không") và với việc kỹ năng mới chỉ đến từ thầy search.
- Hai thất bại khi giữ kỹ năng lúc train RL tiếp (SD7.7 và S2-KL) cùng dùng hệ số KL cố định và neo về chính model cũ. PPO cho thấy β cố định khó chọn; Kickstarting cho thấy λ hằng số giữ học trò quanh mức thầy, còn λ giảm dần mới vượt thầy; Meisheri giữ được kỹ năng bắt chước bằng entropy 0 và clip 0.01. Repo chưa thử ba cách này.
- SDM2 (đoán đối thủ bằng CNN G: +12 vs F nhưng −33 vs RuleBot) là một trường hợp của joint-policy correlation (Lanctot 2017) và của nhận định của Weil 2023 rằng không có mô hình đối thủ đơn nào đúng cho mọi đối thủ. Osogami tránh vấn đề bằng kịch bản bi quan thay vì đoán đúng đối thủ.
- "Hòa cấu trúc" của repo (S3 hòa 53–64% vs CNN) là hiện tượng chung của game có agent an toàn: Osogami (trận gương 78–89%), Perez-Liebana (MCTS hòa 59–84%), Skynet (đối thủ không bom), AlphaZero vs Stockfish (72%). Đóng góp riêng của repo là probe SD9.2 tách được "do luật" khỏi "do agent".
- Về đánh giá, repo làm đúng phần lớn khuyến nghị của Henderson và Agarwal (CI bootstrap theo đơn vị độc lập là seed bàn, cỡ hiệu ứng tối thiểu thay p-value, giao thức cố định, báo FAIL). Chỗ thiếu là biến thiên giữa các lần train: Henderson chỉ ra 5 seed so với 5 seed cùng cấu hình đã khác nhau có ý nghĩa.

10.3 Giới hạn

- Mỗi kết quả chính dựa trên **một train seed** và một seed test; chưa chứng minh độ ổn định qua nhiều lần train (Henderson 2018).
- Các mốc trong biểu đồ mục 8 dùng seed test khác nhau; chỉ so ghép cặp trong cùng một test mới có CI.
- Thầy search giả định đối thủ RuleBot, nên S3 mạnh nhất khi đấu RuleBot; đấu CNN chỉ nhỉnh hơn S2 (+3.33 macro).

- RuleBot là đối thủ duy nhất của train và test chính; chưa có đánh giá với agent bên ngoài (ví dụ agent Pommerman công khai) vì môi trường tự cài khác luật.
- Policy lấy mẫu: tái lập bit-to-bit chỉ trên cùng phiên bản torch; một số lượt do bạn cùng nhóm chạy trên torch 2.11/2.14.
- Kill-search là heuristic: learner chỉ né sau nước đầu, horizon 12, chưa phải MCTS; kế hoạch cần hai quả bom của learner nằm ngoài tầm.
- FFA 3–4 agent và bàn 11×11 mới được đo mô tả (PDE, PDE2, S3-FFA) và một lượt chưng cất 1v1 trên 11×11; chưa train FFA, chưa chạy trên Pommerman chính thức.
- Phần thưởng định hình không phải dạng thế, nên về lý thuyết có thể đổi policy tối ưu.

10.4 Hướng tiếp theo

Thứ tự dưới đây ưu tiên việc rẻ và có căn cứ trực tiếp từ tài liệu.

Hướng	Căn cứ	Vấn đề của repo nó nhắm vào	Chi phí ước tính
Gộp dữ liệu chưng cất qua các vòng (D_SD8A ∪ D_SD9) và lấy ít trạng thái hơn mỗi ván	ExIt: online > batch, một trạng thái mỗi ván; DAgger: Follow-The-Leader trên toàn D	Kiểm tra xem S3 còn cách trần G + search3 1.75 điểm có thu hẹp được không	Thấp: dữ liệu đã có, chỉ train lại
Search dùng prior/value của S3 (MCTS PUCT)	ExIt, AlphaZero: bước cải thiện chuyên gia bằng mạng là nguồn tăng chính	Trần ~94% do search không dùng mạng (SD9.0: nền mạnh hơn không giúp)	Cao: viết search mới, chậm lúc thu
Kịch bản bi quan thay mô hình RuleBot chính xác	Osogami 2019: độ sâu 1 + kịch bản bi quan thắng RL 87.1%	Search đoán sai loại đối thủ (SDM2 −33; SD9.2 bẫy kiểu RuleBot không ăn CNN)	Trung bình: đổi `OPPONENT_MODEL`
Mô hình đối thủ học được hoặc hỗn hợp	DOMAC: dự đoán hành động đối thủ từ quan sát của mình; PSRO: meta-strategy hỗn hợp giảm JPC 29–57%	SD10 ước tính ~53 giờ thu vì gọi CNN thật của đối thủ	Trung bình

Hướng	Căn cứ	Vấn đề của repo nó nhắm vào	Chi phí ước tính
RL tiếp với neo về thầy search, λ giảm dần, entropy 0, clip nhỏ	Kickstarting (λ tuyến tính 1→0 vượt thầy), Meisheri (entropy 0, clip 0.01), PPO (β thích nghi)	D-S2 và S2-KL: RL làm mất kỹ năng chưng cất	Trung bình
SD10 (hướng A, đã cài)	ExIt với thầy đoán đúng đối thủ	S3 hòa 53–64% vs CNN	Cao: ~53 giờ thu
Nhiều train seed cho S2/S3 (≥ 3)	Henderson 2018, Agarwal 2021	Mọi kết luận hiện tại là một lần train	Trung bình
FFA: Phase D/E từ S2/S3 với lấy mẫu đối thủ cũ và cơ chế giữ kỹ năng	Bansal 2018 (đối thủ cũ, ensemble), Huynh 2024	PDE/PDE2 chưa train	Cao

Hai hướng không nên ưu tiên: đổi lịch vòng sớm hơn (SD9.3 đã bác bỏ) và thêm thưởng theo sự kiện; nếu cần shaping thì dùng dạng thế $\gamma\Phi(s') - \Phi(s)$ để không đổi chính sách tối ưu (Ng 1999).

10.5 So sánh với agent đã công bố (25–26/10)

Trong bối cảnh gần Pommerman FFA nhất (11×11, 4 agent), S3-11 thắng **63.8%** và G+s3 thắng **88.8%** khi đấu 3 RuleBot. Các paper công bố 40–43% (PPO) đến 61–78% (MCTS + model học) khi đấu 3 SimpleAgent. Không agent nào được chạy trên cả hai môi trường, nên bảng dưới chỉ định vị, không xếp hạng. Paper đã thêm bảng này (Table VI) và giới hạn (8): chưa chạy trên Pommerman chính thức.

Work	Bối cảnh	Đối thủ	Phương pháp	Kết quả báo cáo
Resnick 2018	Pommerman FFA, 4 agent	3 SimpleAgent	mức tham chiếu	agent ngang SimpleAgent thắng ≈20%
Gao 2019	Pommerman FFA / team	3 SimpleAgent / đội SimpleAgent	PPO + cắt tỉa hành động	WR 40–43% (FFA), 65–76% (team), 3×100 trận

Work	Bối cảnh	Đối thủ	Phương pháp	Kết quả báo cáo
Perez-Liebana 2019	Pommerman FFA, quan sát đầy đủ	3 agent luật	MCTS / RHEA	WR 46.5% / 33.0%
Weil 2023	Pommerman FFA chính thức	3 SimpleAgent	MCTS + model học (kiểu ExIt)	WR 61–78%, hòa 12–34%
Meisheri 2019	team 2×2, quan sát một phần	đội SimpleAgent	bắt chước → PPO	WR 77.8%
Osogami 2019	team 2×2	đội SimpleAgent	tree search độ sâu 1, kịch bản bi quan	WR 92.6–99.7%
Huynh 2024	team 2×2, quan sát một phần	SimpleAgent; agent search dypm	PPO + LSTM, curriculum, self-play quần thể, 200M bước	WR 98.85% vs SimpleAgent; 30.0% vs dypm
Mình, G	FFA 11×11, 4 agent	3 RuleBot	PG có mask + advantage nhóm	WR 36.3%, ActiveWR 23.1% (160 trận)
Mình, G+s3	FFA 11×11, 4 agent	3 RuleBot	G + kill-search không dùng mạng	WR 88.8%, ActiveWR 86.3% (160 trận)
Mình, S3-11	FFA 11×11, 4 agent	3 RuleBot	CNN chưng cất, không search	WR 63.8%, ActiveWR 59.4% (160 trận)
Mình, S3	FFA 9×9, 3 agent	2 RuleBot	CNN chưng cất, không search	WR 87.8%, ActiveWR 82.8% (180 trận)
Mình, G+s3	1v1 9×9	RuleBot	G + kill-search	WR 99.5%, ActiveWR 94.0% (400 trận)
Mình, S3	1v1 9×9	RuleBot	CNN chưng cất	WR 98.0%, ActiveWR 92.25% (400 trận)

Mọi số của paper khác đều đối chiếu với toàn văn đã lưu. Goulart 2019 bị loại vì chỉ đọc được abstract, không kiểm được con số 61.02%.

Môi trường nào khó hơn. Nói chung Pommerman khó hơn: bàn 11×11, 4 agent, có đá bom, team mode quan sát một phần. Tuy vậy có ba lý do khiến số của mình là bảo thủ, và paper đã nhấn mạnh cả ba:

1. **Tiêu chí chặt hơn.** Mọi paper đếm cả trận đối thủ tự chết; ActiveWR chỉ đếm trận agent tự giết. Theo tiêu chí của họ, S3 là 98.0 chứ không phải 92.25, và G+s3 FFA là 88.8 chứ không phải 86.3.
2. **Không được lợi khi chờ.** Bom nổ sau 8 bước thay vì 10, và vòng co ép cuối trận. Trong khi đó ở Pommerman, phần lớn trận hòa là do chạm giới hạn 800 bước (Weil 2023).
3. **Ngân sách nhỏ.** S3 chạy một lượt mạng mỗi nước và được train trên máy cá nhân. Agent học mạnh nhất của Pommerman cần 200 triệu bước mà vẫn chỉ thắng 30% trước một tree search.

Thứ tự cũng giống với các paper Pommerman: search dẫn trước học, và giữa các agent mạnh thì hòa chiếm ưu thế. RuleBot và SimpleAgent đều là heuristic yếu, dưới mức chia đều (11.9% và khoảng 20%), nhưng chưa từng được cho đấu với nhau. Việc kiểm trực tiếp nhất là chạy S3 và G+s3 trên Pommerman chính thức; việc này chưa làm vì tốn công chuyển môi trường và train lại.

11. Paper IEEE và biểu đồ (24–26/10)

Toàn bộ kết quả đã được viết thành paper IEEE conference `paper/main.tex`: **14 trang, 15 hình, 6 bảng**, biên dịch sạch bằng tectonic. Paper giữ đoạn khai báo có dùng AI. Mọi hình được vẽ lại từ log từng trận trong `runs/` bằng `paper/scripts/make_figures.py`, và số liệu ghi vào `paper/figures/figure_data.json`.

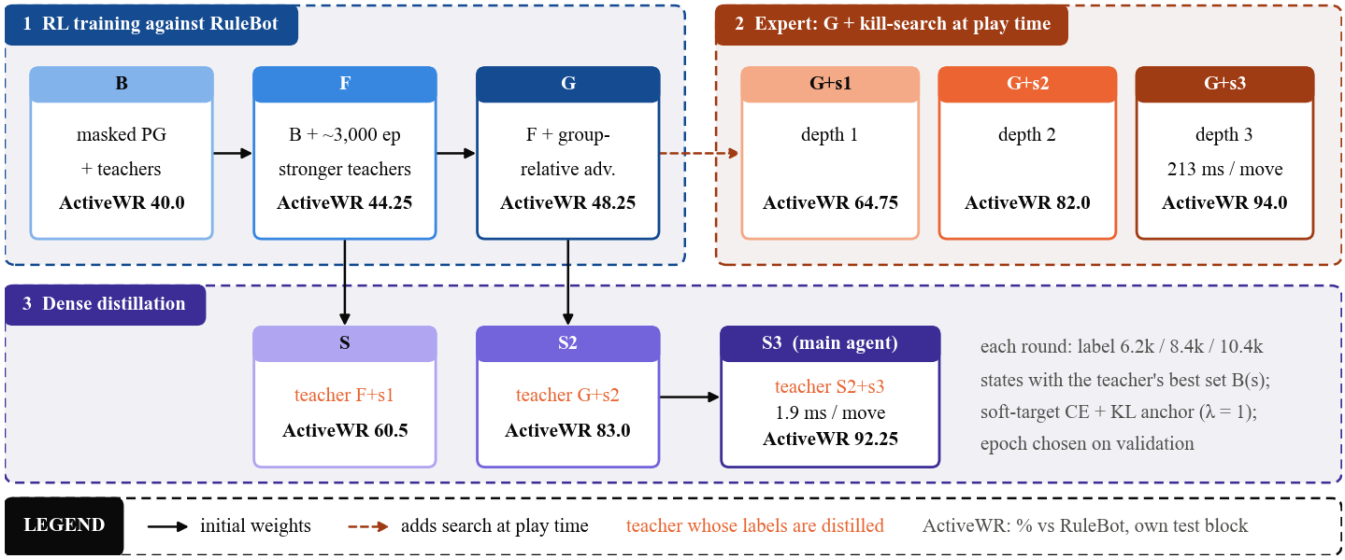
Hình	Nội dung	Vai trò trong paper
1 Pipeline	B → F → G (RL), G+s1/s2/s3 (search), S → S2 → S3 (chưng cất), ActiveWR từng mốc	Tóm tắt cả phương pháp
2 Gameplay	4 khung hình thật: S3 ép RuleBot bằng hai quả bom	Minh họa một pha giết chủ động
3 Kiến trúc	CNN 3 lớp conv → FC 256 → actor/critic, 1 388 359 tham số	Mô tả mạng

Hình	Nội dung	Vai trò trong paper
4 Forest	15 hiệu ghép cặp, chia 5 nhóm, có ngưỡng +5	Gom mọi so sánh, kể cả kết quả âm
5 Chưng cất	loss, độ khớp với thầy, KL neo, validation của 3 vòng	Cơ chế chưng cất
6, 7 Mốc và thành phần kết quả	ActiveWR 9 mốc; tỉ lệ thắng chủ động / thụ động / hòa / thua	Tiến trình 40 → 92.25 và bằng chứng cho ActiveWR
8 WR so với ActiveWR	4 can thiệp làm WR tăng mà ActiveWR đứng yên	Bằng chứng trực tiếp nhất cho ActiveWR
9 Confusion	hành động của học trò so với thầy, trước và sau mỗi vòng	Chưng cất sửa cách di chuyển
10 Độ sâu search	ActiveWR theo độ sâu 0–3	Search sâu hơn thì mạnh hơn
11, 12 Đấu CNN và trận hòa	đấu CNN phần lớn là hòa; ít trận hòa có kế hoạch giết	Nêu và giải thích điểm yếu
13–15 Phụ lục	13 kênh quan sát, độ dài ván, đường train RL	Chi tiết kỹ thuật

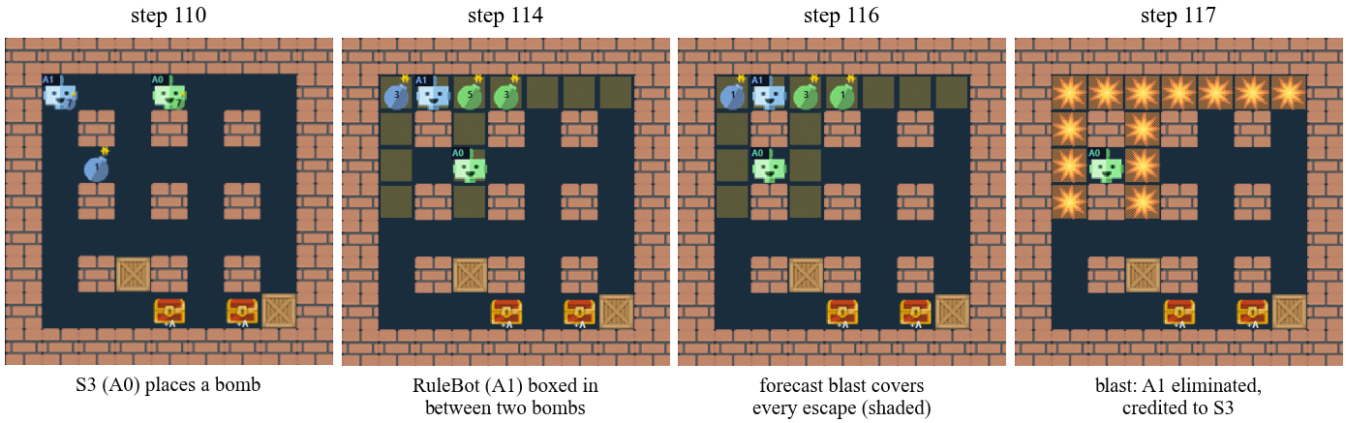
11.1 Các hình của paper

Ảnh là bản PNG đúng với PDF trong paper sau khi đã sửa ngày 26/10. Chi tiết nằm trong chú thích của `paper/main.tex`.

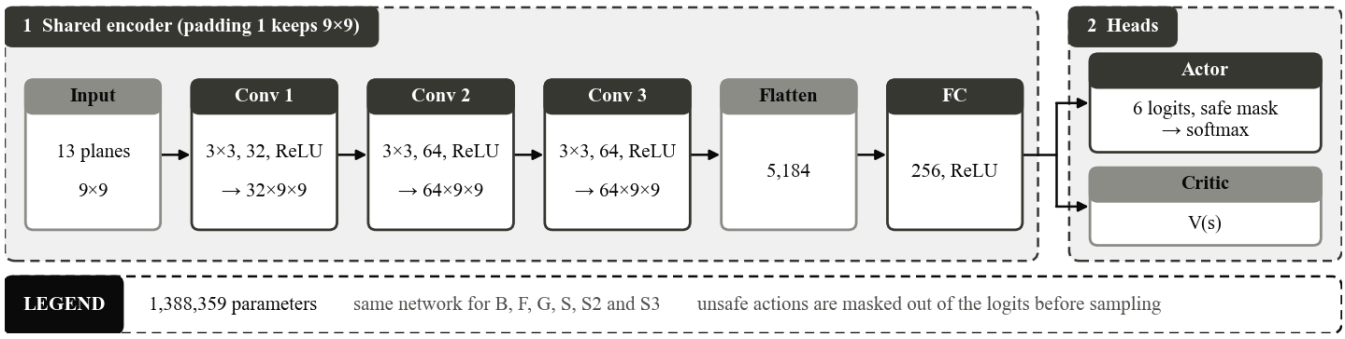
Hình 1. Pipeline từ RL (B → F → G) sang search lúc chơi (G+s1/s2/s3) và chưng cất dày (S → S2 → S3), kèm ActiveWR từng mốc.



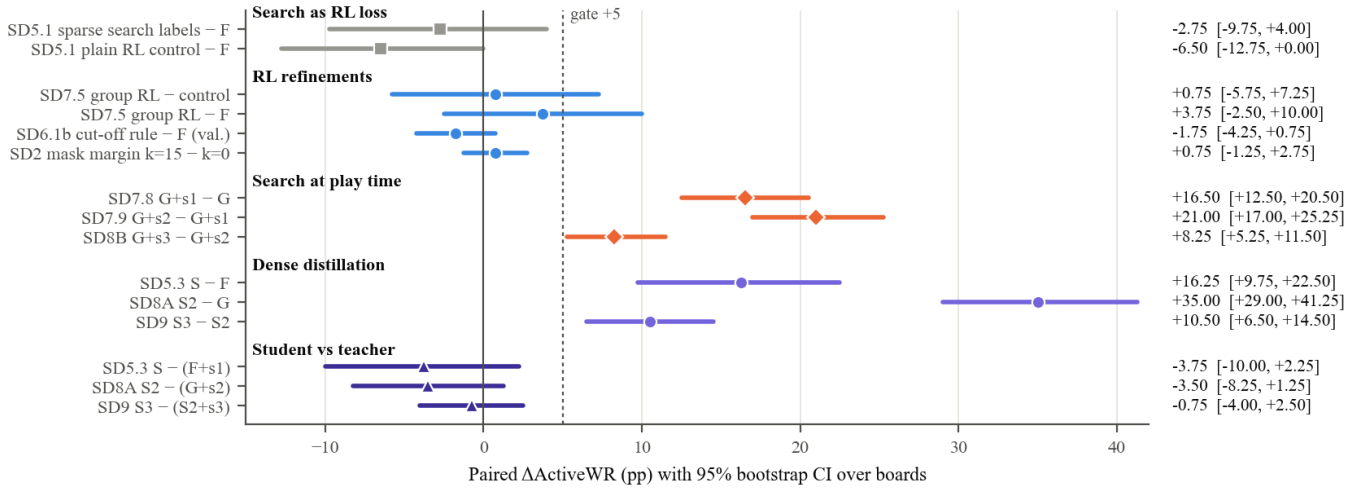
Hình 2. Bốn khung hình thật trên bàn validation 21000: S3 đặt hai quả bom ép RuleBot vào góc, vụ nổ được tính cho S3.



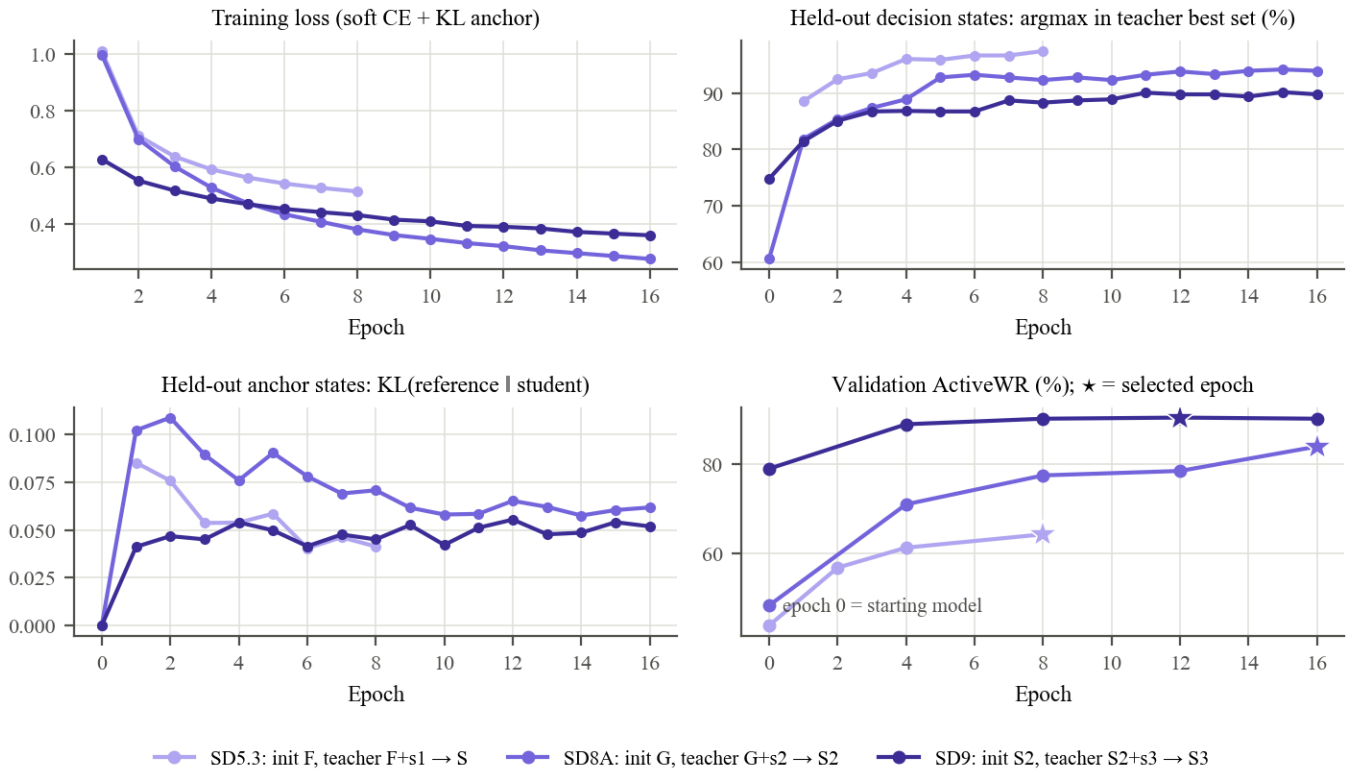
Hình 3. Kiến trúc CNN dùng chung cho mọi model: 3 lớp conv 3×3, FC 256, đầu actor (6 hành động, có mask an toàn) và critic; 1 388 359 tham số.



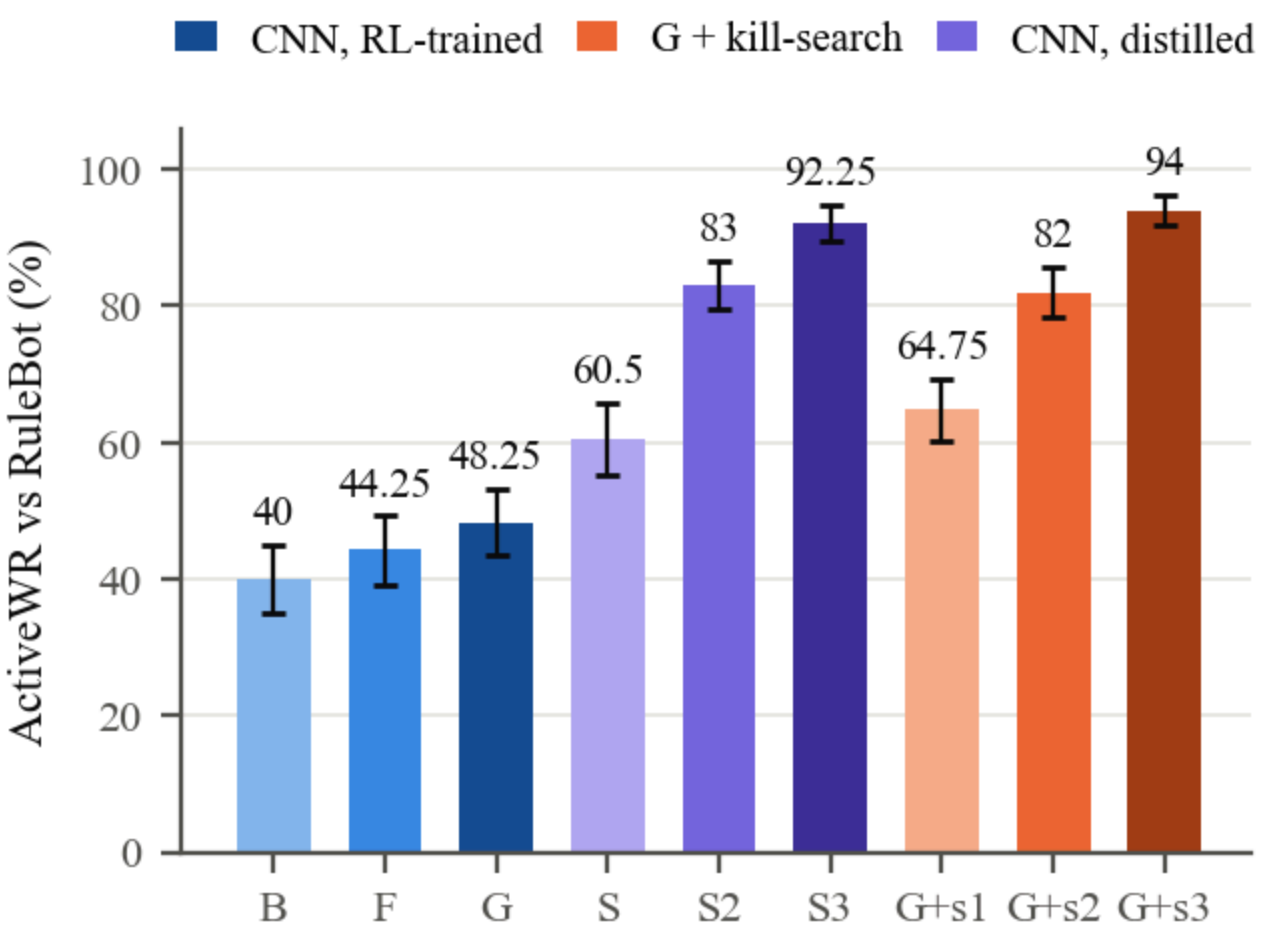
Hình 4. Forest plot 15 hiệu ActiveWR ghép cặp với CI95, có đường ngưỡng +5. Nhóm dưới cùng so học trò với đúng thầy của nó (S3 − (S2+s3) = −0.75).



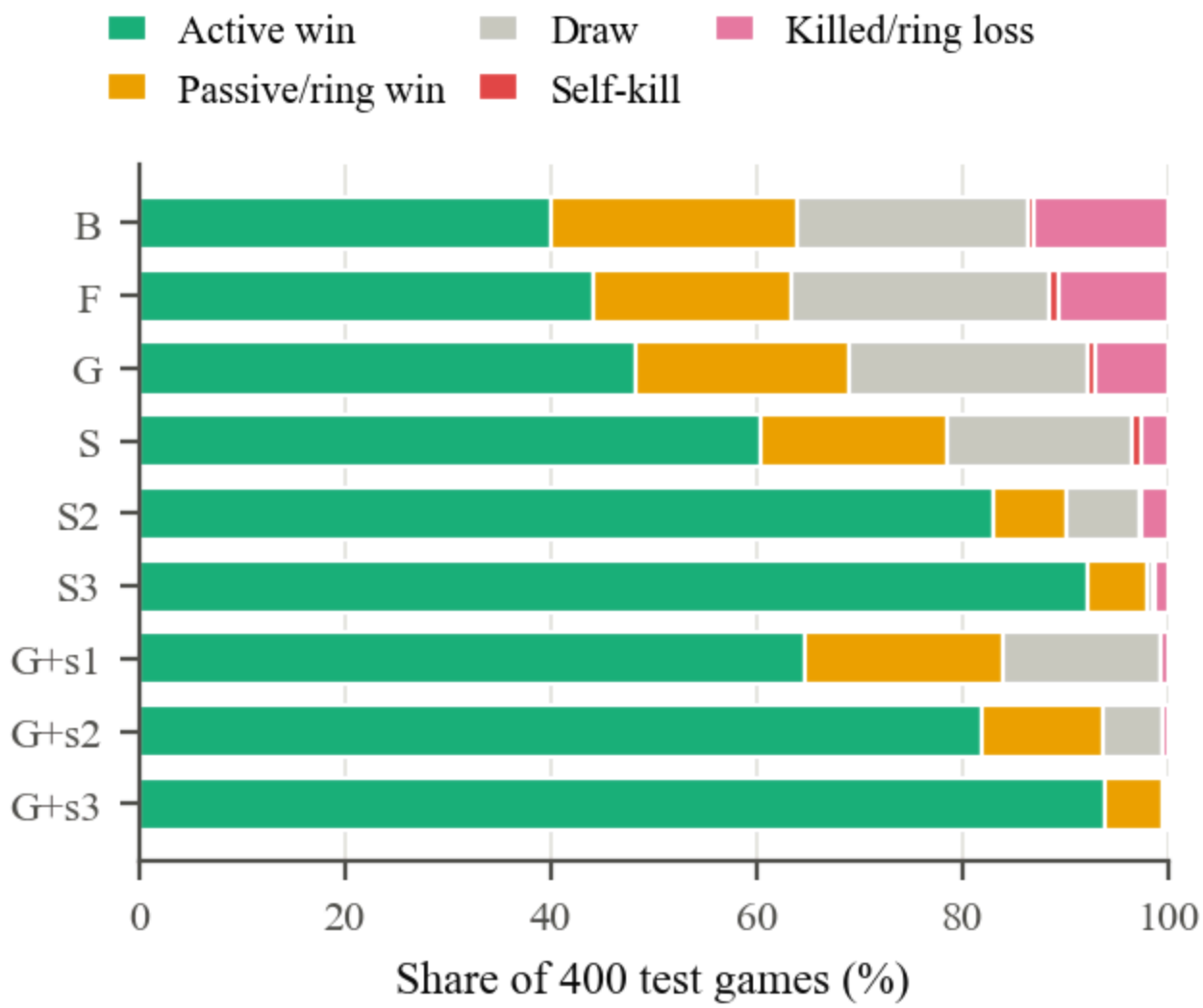
Hình 5. Ba vòng chưng cất: loss, độ khớp với thầy trên trạng thái giữ lại, KL neo, ActiveWR validation (ngôi sao là epoch được chọn).



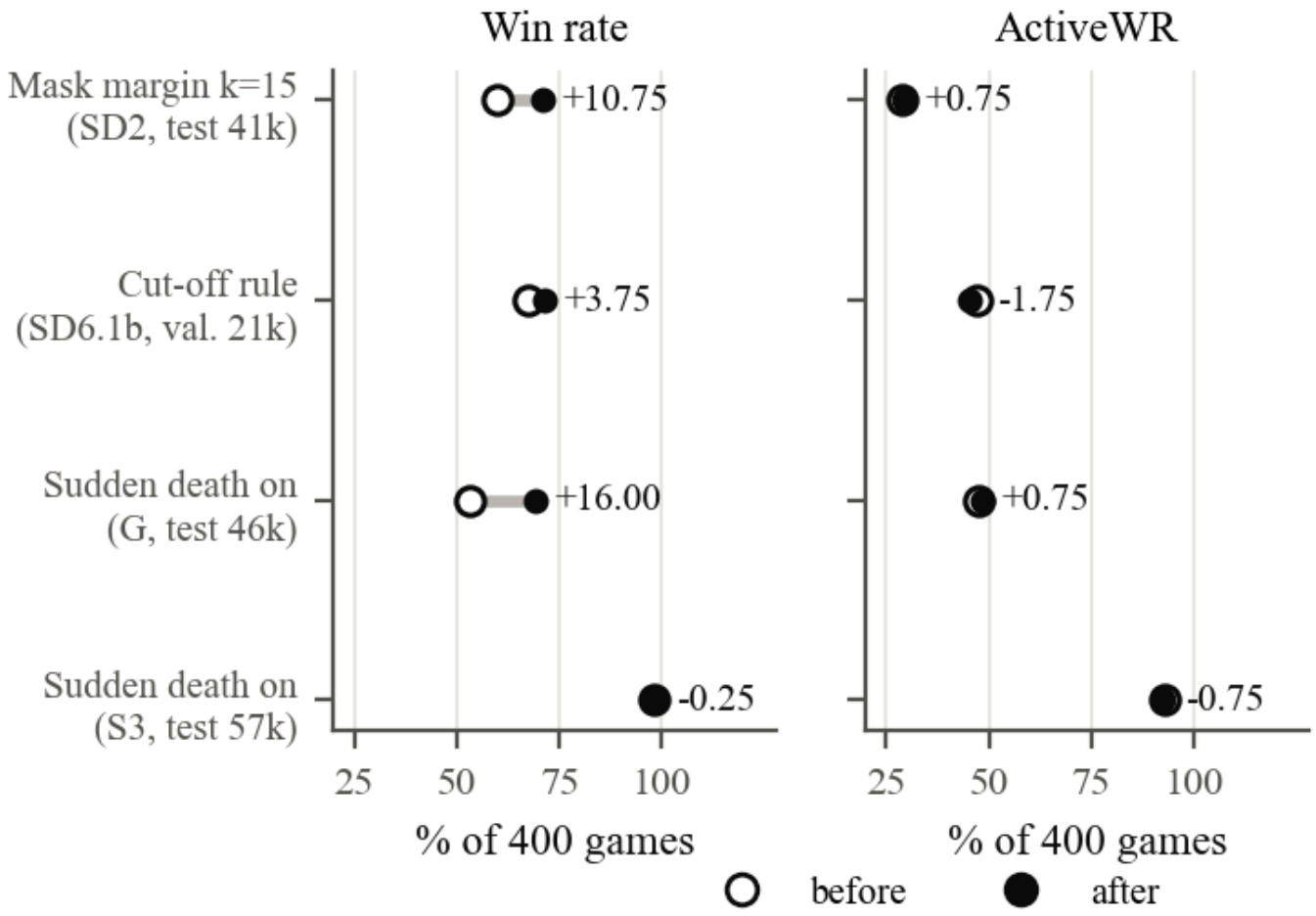
Hình 6. ActiveWR của 9 mốc, mỗi mốc 400 trận trên block test riêng, nên so giữa các cột chỉ mang tính mô tả.



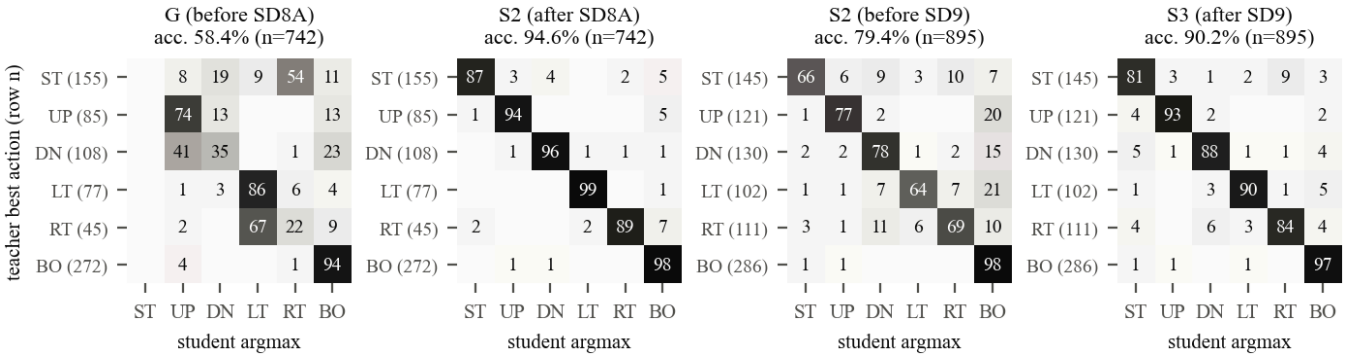
Hình 7. Thành phần kết quả của cùng các trận đó: checkpoint RL thắng nhiều nhờ đối thủ tự chết hoặc chết vì vòng; agent chưng cất và agent search thắng bằng cách giết.



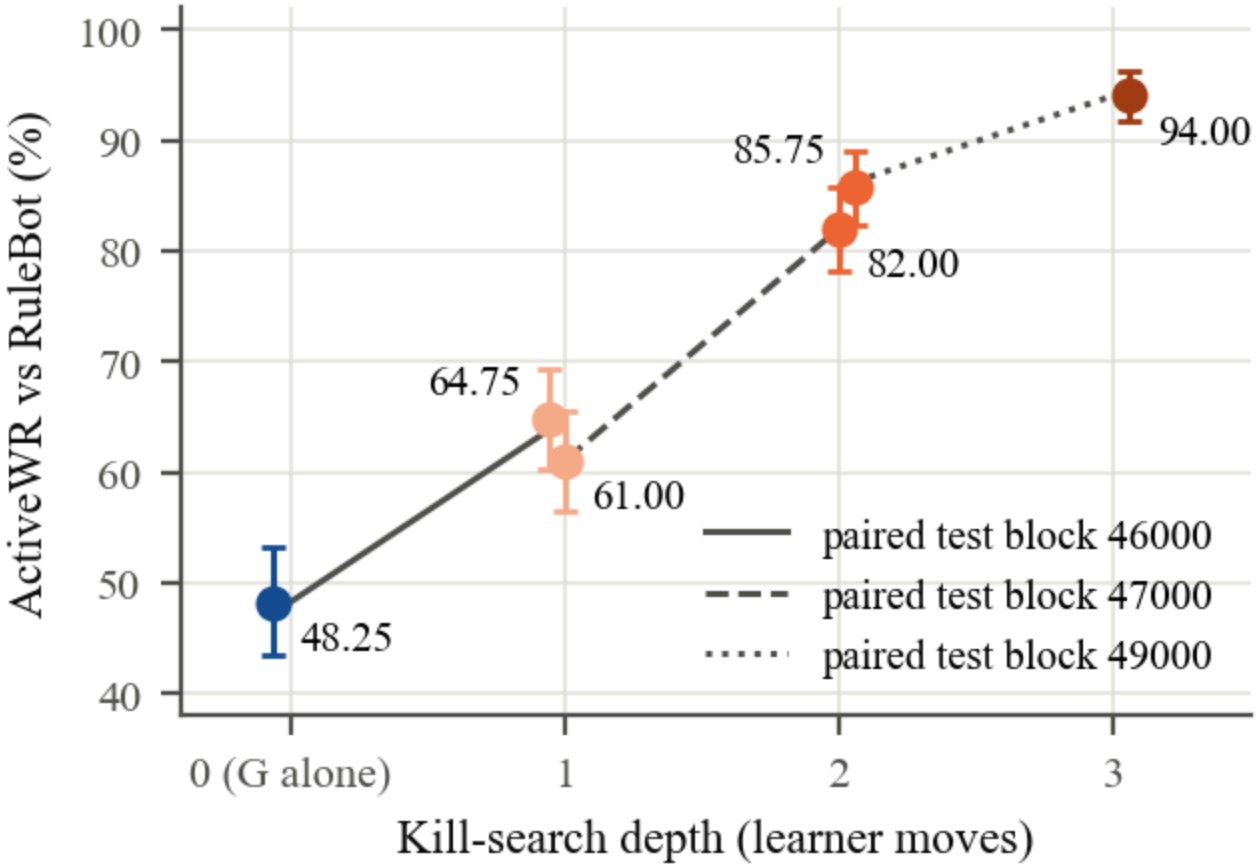
Hình 8. Bốn can thiệp đo bằng cả hai chỉ số: WR tăng 3.75–16 điểm trong khi ActiveWR gần như không đổi.



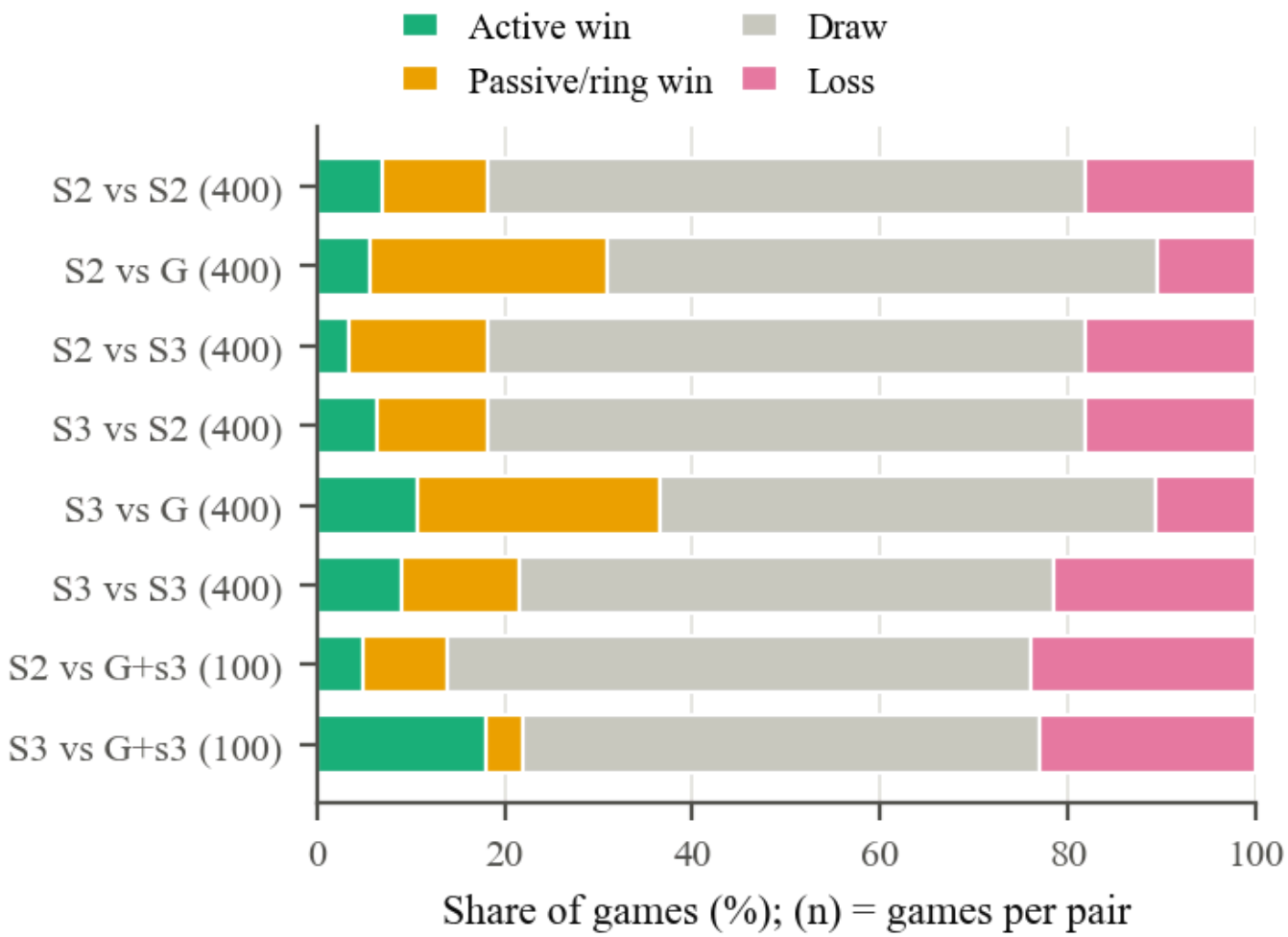
Hình 9. Ma trận nhầm hành động (hàng: nước tốt nhất của thầy; cột: argmax của học trò) trước và sau vòng 2 và vòng 3. Chưng cất chủ yếu sửa hướng đi; BOMB đã đúng từ trước.



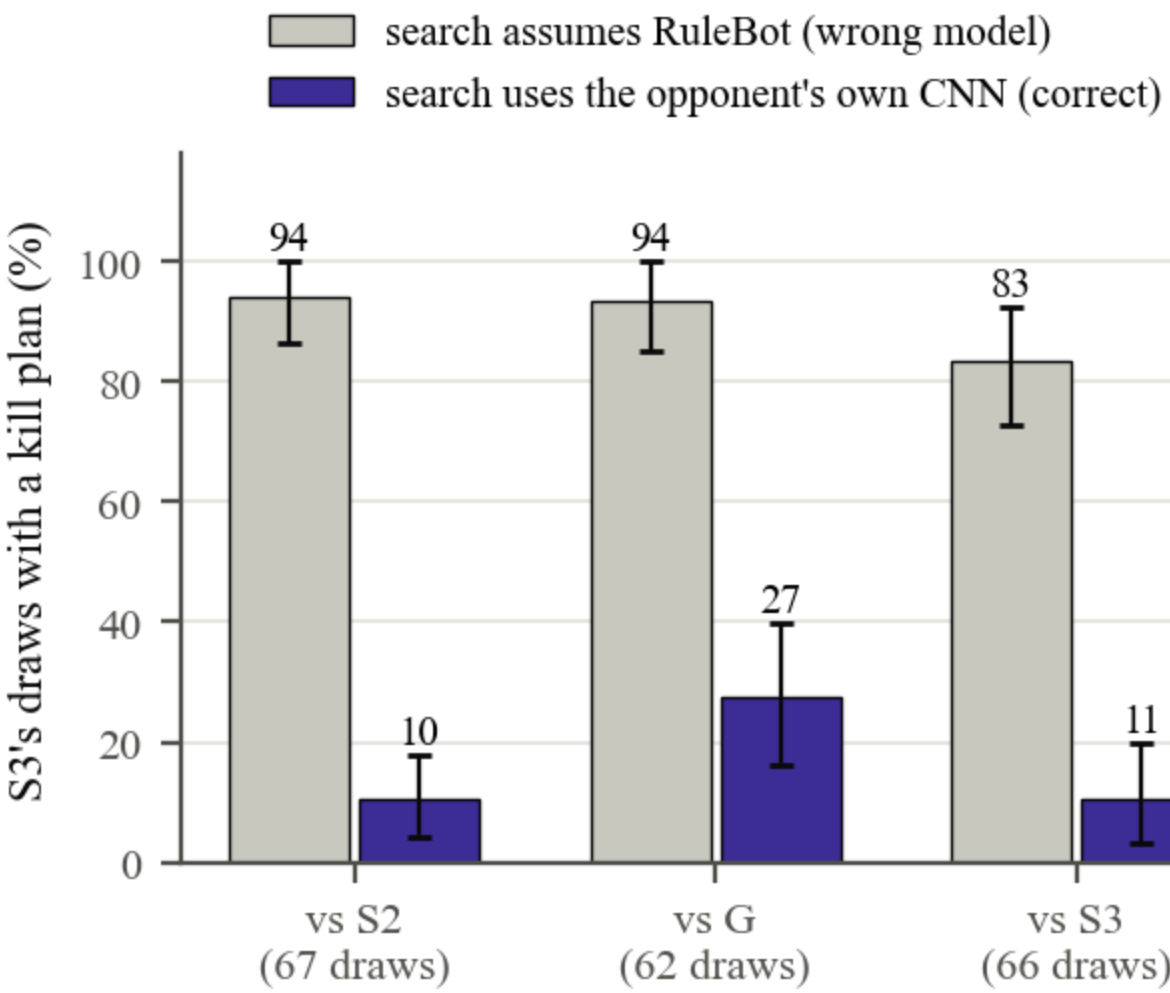
Hình 10. ActiveWR theo độ sâu kill-search quanh G. Mỗi kiểu nét nối hai agent đo trên cùng một block ghép cặp: 48.25 → 64.75, 61.0 → 82.0, 85.75 → 94.0.



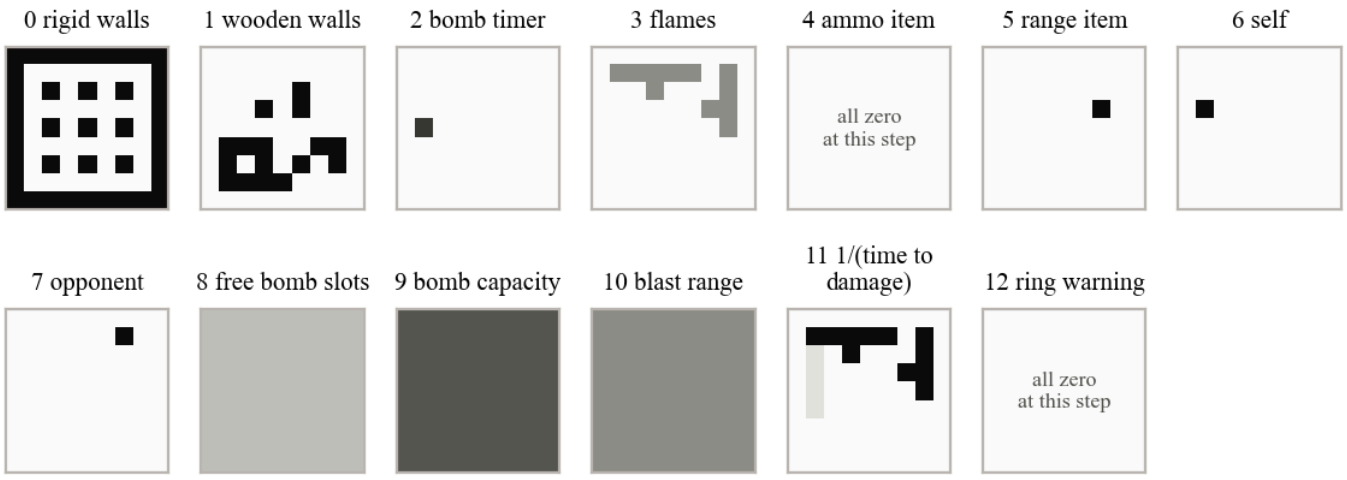
Hình 11. Kết quả khi đấu CNN và agent search trên bàn validation: phần lớn là hòa; phần thắng thụ động chủ yếu do đối thủ bị vòng bắt.



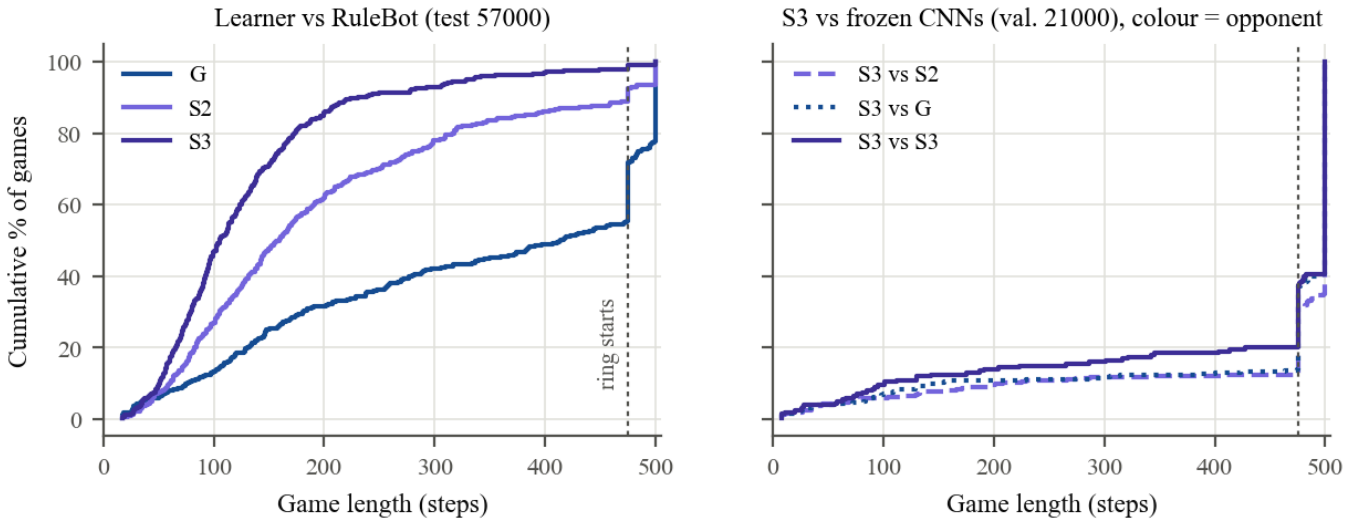
Hình 12. Tỉ lệ trận hòa của S3 mà search độ sâu 3 tìm được kế hoạch giết. Đoán đối thủ là RuleBot (sai) thì tìm được 83–94%; dùng CNN thật của đối thủ thì chỉ 10–27%.



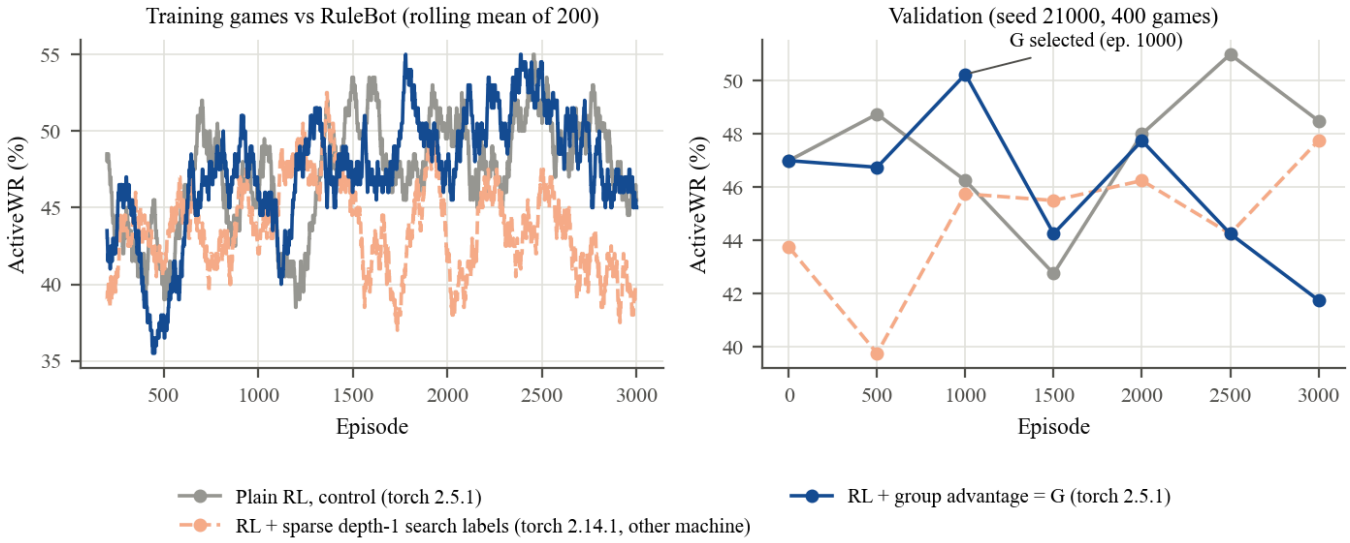
Hình 13. 13 kênh quan sát của một agent (bàn 21007, bước 26, RuleBot đấu RuleBot; đậm hơn = lớn hơn).



Hình 14. Phân phối tích lũy độ dài ván. Đầu RuleBot, agent mạnh hơn kết thúc sớm hơn (trung vị 414 / 158 / 105 bước cho G / S2 / S3). Đầu CNN, 59.5–65% trận của S3 chạm 500 bước.



Hình 15. RL từ F (3 000 episode mỗi nhánh, một train seed). Nhánh advantage nhóm đạt đỉnh ở episode 1000 và trở thành G; khi test, nó chỉ hơn control +0.75.



11.2 Bổ sung, kiểm lại hình và watermark

Paper có thêm mục FFA (mục 7.9 ở trên), Table VI so sánh với các paper khác (mục 10.5) và câu "mức chia đều 33% / 25%, 94% / 93% trận thắng của S3 và S3-11 là chủ động so với 64% / 58% của G", để biến FFA thành bằng chứng cho ActiveWR.

Kiểm lại hình (26/10). Hình 1 và 3 được vẽ lại theo kiểu nhóm nét đứt, có assert chặn chữ tràn ra ngoài ô. Sau đó toàn bộ số trên 12 hình được đối chiếu với log gốc, phát hiện và sửa hai lỗi:

- **Thầy của S3 là S2+s3, không phải G+s3.** So đúng với thầy trên cùng block 57000, S3 chỉ kém **0.75 điểm [−4.0, +2.5]**. Con số −1.75 [−5.0, +1.5] là khoảng cách tới G+s3, agent mạnh nhất (trần). Khoảng cách trò–thầy qua ba vòng vì vậy là −3.75 → −3.5 → −0.75. Đã

sửa hình 4 và 5 câu trong paper; không chạy test mới.

- **Chú thích hình 13 tả sai ảnh.** Ảnh là bước 26 chứ không phải 25; agent chứa được 2 bom và đang đặt 1; trên bàn có vật phẩm tầm nổ. Ảnh trước đây không tái lập được vì RuleBot chọn ngẫu nhiên khi các nước ngang điểm; đã cố định seed.
- Các hình còn lại khớp log. Đã sửa thêm ba chỗ chữ đè nhau (hình 4, 10, 13).

Kiểm watermark (26/10). Quy trình dùng hai skill watermarks-remover, chạy qua service local. Kết quả quan sát:

- 37 file (`.tex` , PDF, PNG) không có C2PA, không có metadata AI, 0 ký tự Unicode ẩn.
- Chỉ gỡ dòng `Software: Matplotlib` trong PNG.
- Phần chữ có điểm stylometry 0.03 (mức low), nên không viết lại.

Service không có exiftool hay bộ dò watermark thống kê, nên kết quả chỉ nói về metadata và Unicode.

Còn mở: tác giả, funding, CRediT và giới hạn số trang của hội nghị (paper hiện 14 trang).

Tài liệu tham khảo

Tra cứu ngày 25/10/2026; thông tin lấy từ trang arXiv/nhà xuất bản. Số liệu của repo lấy từ `RESULTS.md` , `IMPROVEMENTS.md` , `INSIGHTS.md` , `BAO_CAO_TONG_KET.md` và mã nguồn (`train_actor_critic.py` , `kill_search.py` , `group_rl.py` , `run_sd53_pilot.py` , `run_sd9_distill.py` , `pilot_two_bomb.py`).

Pommerman / Bomberman

1. Resnick, C. et al. (2018). Pommerman: A Multi-Agent Playground. arXiv:1809.07124.
2. Resnick, C., Raileanu, R., Kapoor, S., Peysakhovich, A., Cho, K., Bruna, J. (2018). Backplay: "Man muss immer umkehren". arXiv:1807.06919.
3. Peng, P., Pang, L., Yuan, Y., Gao, C. (2018). Continual Match Based Training in Pommerman. arXiv:1812.07297.
4. Gao, C., Hernandez-Leal, P., Kartal, B., Taylor, M. E. (2019). Skynet: A Top Deep RL Agent in the Inaugural Pommerman Team Competition. RLDM 2019; arXiv:1905.01360.
5. Gao, C., Kartal, B., Hernandez-Leal, P., Taylor, M. E. (2019). On Hard Exploration for Reinforcement Learning: a Case Study in Pommerman. AIIDE 2019.
6. Osogami, T., Takahashi, T. (2019). Real-time tree search with pessimistic scenarios: Winning the NeurIPS 2018 Pommerman Competition. ACML 2019, PMLR 101; arXiv:1902.10870.

7. Perez-Liebana, D., Gaina, R. D., Drageset, O., İlhan, E., Balla, M., Lucas, S. M. (2019). Analysis of Statistical Forward Planning Methods in Pommerman. AIIDE 2019, 15(1), 66–72.
8. Weil, J., Czech, J., Meuser, T., Kersting, K. (2023). Know your Enemy: Investigating Monte-Carlo Tree Search with Opponent Models in Pommerman. ALA Workshop @ AAMAS 2023.
9. Huynh, N.-M., Cao, H.-G., Wu, I-C. (2024). Multi-Agent Training for Pommerman: Curriculum Learning and Population-based Self-Play Approach. arXiv:2407.00662.
10. Meisheri, H., Shelke, O., Verma, R., Khadilkar, H. (2019). Accelerating Training in Pommerman with Imitation and Reinforcement Learning. arXiv:1911.04947.
11. Goulart, Í., Paes, A., Clua, E. (2019). Learning How to Play Bomberman with Deep Reinforcement and Imitation Learning. ICEC-JCSG 2019, LNCS 11863, 121–133. (Chưa đọc được toàn văn: HAL chặn tải tự động.)
12. Sun, J., Chen, S., Zhang, C., Ma, Y., Zhang, J. (2022). Decision-making with Speculative Opponent Models (DOMAC). arXiv:2211.11940.
13. Helfenstein, F., Czech, J., Blüml, J., Eisel, M., Kersting, K. (2024). Checkmating One, by Using Many: Combining Mixture of Experts with MCTS (M2CTS). arXiv:2401.16852.

Search và học

14. Kartal, B., Hernandez-Leal, P., Taylor, M. E. (2018). Using Monte Carlo Tree Search as a Demonstrator within Asynchronous Deep RL. arXiv:1812.00045.
15. Kartal, B., Hernandez-Leal, P., Gao, C., Taylor, M. E. (2019). Safer Deep RL with Shallow MCTS: A Case Study in Pommerman. arXiv:1904.05759.
16. Kartal, B., Hernandez-Leal, P., Taylor, M. E. (2019). Action Guidance with MCTS for Deep Reinforcement Learning. AIIDE 2019.
17. Anthony, T., Tian, Z., Barber, D. (2017). Thinking Fast and Slow with Deep Learning and Tree Search. NeurIPS 2017.
18. Silver, D. et al. (2017). Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm. arXiv:1712.01815.
19. Ross, S., Gordon, G., Bagnell, D. (2011). A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning. AISTATS 2011.
20. Rusu, A. A. et al. (2015). Policy Distillation. arXiv:1511.06295.
21. Schmitt, S. et al. (2018). Kickstarting Deep Reinforcement Learning. arXiv:1803.03835.

Tối ưu policy, an toàn, đối thủ

22. Sutton, R. S., Barto, A. G. (2018). Reinforcement Learning: An Introduction, 2nd ed. MIT Press. (Đọc chương 13, Policy Gradient Methods.)
23. Williams, R. J. (1992). Simple statistical gradient-following algorithms for connectionist reinforcement learning. Machine Learning 8, 229–256.
24. Mnih, V. et al. (2016). Asynchronous Methods for Deep Reinforcement Learning. ICML 2016.
25. Schulman, J. et al. (2015). High-Dimensional Continuous Control Using Generalized Advantage Estimation. ICLR 2016.
26. Schulman, J. et al. (2017). Proximal Policy Optimization Algorithms. arXiv:1707.06347.
27. Shao, Z. et al. (2024). DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models (GRPO). arXiv:2402.03300.
28. Ng, A. Y., Harada, D., Russell, S. (1999). Policy Invariance Under Reward Transformations: Theory and Application to Reward Shaping. ICML 1999, 278–287.
29. Alshiekh, M. et al. (2018). Safe Reinforcement Learning via Shielding. AAAI 2018.
30. Huang, S., Ontañón, S. (2022). A Closer Look at Invalid Action Masking in Policy Gradient Algorithms. FLAIRS-35.
31. Bansal, T. et al. (2018). Emergent Complexity via Multi-Agent Competition. ICLR 2018.
32. Lanctot, M. et al. (2017). A Unified Game-Theoretic Approach to Multiagent Reinforcement Learning (PSRO). NeurIPS 2017.

Đánh giá

33. Henderson, P. et al. (2018). Deep Reinforcement Learning that Matters. AAAI 2018.
34. Agarwal, R., Schwarzer, M., Castro, P. S., Courville, A., Bellemare, M. G. (2021). Deep Reinforcement Learning at the Edge of the Statistical Precipice. NeurIPS 2021.
35. Efron, B. (1979). Bootstrap methods: another look at the jackknife. The Annals of Statistics 7(1), 1–26. doi:10.1214/aos/1176344552.